VitalLink

Backend System Report

System analysis · architecture · database (ER) · class design
processes · API · security · testing · deployment · status

Kathmandu University — Capstone Project, Group 2

Amrit Gautam · Dinesh Chaudhary · Shreenibash Shrestha · Sudip Ghorasaini

Owner: Abhinaash

Version 1.0 · 15 October 2026

Status: phases 1–5b built (18 of 35 tables, 44 operations, 134 tests)

Document control

Version	Date	Author	Change
1.0	15 Oct 2026	Group 2	Complete report of the backend as built through phase 5b

Contents

1 Introduction 9
1.1 Purpose of this document 9
1.2 Scope 9
1.3 Intended readers 9
1.4 Definitions and acronyms 9
1.5 References 10
2 System analysis 11
2.1 Problem statement 11
2.2 Objectives 11
2.3 Stakeholders and actors 11
2.4 Use case model 12
2.5 Functional requirements 15
2.6 Non-functional requirements 16
2.7 Feasibility study 16
2.8 Constraints and assumptions 16
2.9 Data flow diagrams 17
3 Architecture 20
3.1 Why a modular monolith 22
3.2 Layers inside the service 23
3.3 Modules 24
3.4 Shared core 25
3.5 Technology stack 25
3.6 Source layout 25
4 Database design 27
4.1 Design rules 27
4.2 Entity–relationship diagrams 27
4.3 Relationship summary 33
4.4 Data dictionary 34
4.4.1 users 34
4.4.2 profiles 34
4.4.3 consents 35
4.4.4 audit_log 36
4.4.5 metrics 37
4.4.6 health_readings 38
4.4.7 ingest_batches 38
4.4.8 devices 39
4.4.9 exercises 39
4.4.10 workouts 39

4.4.11 exercise_sets ... 40
4.4.12 training_plans ... 41
4.4.13 plan_sessions ... 42
4.4.14 foods ... 42
4.4.15 meal_entries ... 43
4.4.16 meal_items ... 43
4.4.17 hydration_logs ... 44
4.4.18 nutrition_targets ... 44
4.5 Enumerated types ... 45
4.6 Migrations ... 46
4.7 Capacity estimate ... 46
5 Class design ... 47
5.1 Domain model: accounts, sharing and audit ... 47
5.2 Domain model: health, training and nutrition ... 49
5.3 Service layer ... 51
5.4 Consent engine ... 53
6 Processes in detail ... 56
6.1 Request lifecycle ... 56
6.2 Sign-in, token verification and account creation ... 58
Token verification ... 60
Account creation (insert-if-absent) ... 60
6.3 Phone sync (ingest) ... 60
6.4 Manual readings ... 63
6.5 Time series ... 63
6.6 Dashboard, assessments and readiness ... 63
6.7 Devices ... 64
6.8 Sharing and the consent decision ... 64
Creating a share ... 66
Deciding a shared read ... 66
Why the audit write has its own transaction ... 68
Revocation and the access log ... 69
6.9 Training ... 69
The training view ... 69
6.10 Nutrition ... 71
6.11 Planned processes ... 73
7 API specification ... 77
7.1 Conventions ... 77
7.2 Status and error codes ... 77
7.3 Endpoint reference ... 78
7.4 Request and response bodies ... 80

7.5 Contract with the frontend ... 85
8 Security and privacy ... 86
8.1 Database lockdown ... 86
8.2 Authentication and authorisation ... 86
8.3 Threats and controls ... 86
8.4 Privacy commitments and legal context ... 87
9 Testing and quality assurance ... 88
9.1 Test environment ... 88
9.2 Test suites ... 88
9.3 Guardrails ... 89
9.4 Continuous integration ... 90
9.5 Manual verification ... 91
10 Deployment and operations ... 92
10.1 Environments ... 93
10.2 Configuration ... 93
10.3 Running locally, step by step ... 93
10.4 Container ... 93
10.5 Production rollout (pending) ... 94
10.6 Observability and free-tier limits ... 94
11 Implementation status and plan ... 95
11.1 Work completed ... 95
11.2 Build plan ... 95
11.3 Risks ... 97
11.4 Open decisions ... 97
Appendix A Metric catalogue ... 99
Appendix B Exercise and food catalogues ... 101
Appendix C Sample requests ... 103
C.1 Sync upload ... 103
C.2 Create a share and read through it ... 103
C.3 Log a meal ... 103

List of figures

Figure 1. Use case diagram: owners, viewers, the phone, the identity provider and the scheduler
Figure 2. Data flow diagram, level 0 (context)
Figure 3. Data flow diagram, level 1: six processes and six data stores
Figure 4. System context: clients, identity provider, API, database, storage and scheduler
Figure 5. Layers a request passes through
Figure 6. Module dependencies (arrows point to what a module calls)
Figure 7. ER diagram, as built: all 18 tables and their relationships
Figure 8. ER detail: accounts, sharing and the audit log
Figure 9. ER detail: health readings, ingest batches, devices and the metric catalogue
Figure 10. ER detail: training
Figure 11. ER detail: nutrition
Figure 12. ER diagram, planned: the 17 tables of phases 5c–8
Figure 13. Class diagram: User, Profile, Consent, AuditEntry
Figure 14. Class diagram: health, training and nutrition entities
Figure 15. Class diagram: dependencies and services
Figure 16. Class diagram: the consent engine
Figure 17. Sequence: life of any API request
Figure 18. Sequence: sign-in and first request
Figure 19. Sequence: phone sync upload
Figure 20. Decision ladder: how an upload is accepted or refused
Figure 21. Sequence: creating a share, a shared read, and revocation
Figure 22. Decision ladder: the eight consent rules
Figure 23. State diagram: the life of a share
Figure 24. Sequence: logging a synced workout
Figure 25. Activity: personal record detection
Figure 26. State diagram: training plan status
Figure 27. Sequence: food search, meal logging and the day view
Figure 28. Sequence (planned): document upload with signed URLs
Figure 29. State diagram (planned): a record's file status
Figure 30. Sequence (planned): the nightly job
Figure 31. CI/CD pipeline
Figure 32. Deployment diagram (dashed: production, pending accounts)
Figure 33. Build plan: phases and dependencies

List of tables

Table 1. Suggested reading order
Table 2. Definitions and acronyms
Table 3. Actors
Table 4. UC-01: Sign in and create account
Table 5. UC-02: Edit profile, settings and daily targets
Table 6. UC-03: Sync readings and devices
Table 7. UC-04: View dashboard and series
Table 8. UC-05: Log workouts and plans
Table 9. UC-06: Log meals, foods and water
Table 10. UC-07: Share data with a person
Table 11. UC-08: Revoke a share
Table 12. UC-09: Read the access log
Table 13. UC-10: Read data shared with me
Table 14. Functional requirements
Table 15. Non-functional requirements
Table 16. Feasibility
Table 17. Level 1 processes
Table 18. Architecture options considered
Table 19. Layer responsibilities
Table 20. Modules and table ownership
Table 21. Core components
Table 22. Technology stack
Table 23. Schema-wide rules
Table 24. Relationships
Table 25. PostgreSQL enums (live)
Table 26. Migration history
Table 27. Capacity
Table 28. Blood pressure categories (checked top to bottom)
Table 29. Other planned processes
Table 30. API conventions
Table 31. Status and error codes
Table 32. Operations endpoints
Table 33. Account endpoints
Table 34. Health endpoints
Table 35. Sharing (owner side) endpoints
Table 36. Shared views (viewer side, consent-checked) endpoints
Table 37. Training endpoints
Table 38. Nutrition endpoints
Table 39. Schema IngestIn
Table 40. Schema BucketIn
Table 41. Schema DeviceIn
Table 42. Schema IngestOut
Table 43. Schema ManualIn
Table 44. Schema ReadingOut
Table 45. Schema MetricSummary
Table 46. Schema Series
Table 47. Schema AccountPatch
Table 48. Schema TargetsPatch
Table 49. Schema ShareIn
Table 50. Schema Share

Table 51. Schema SharedWithMe
Table 52. Schema AccessLogEntry
Table 53. Schema WorkoutIn
Table 54. Schema SetIn
Table 55. Schema TrainingWeek
Table 56. Schema MealIn
Table 57. Schema ItemIn
Table 58. Schema NutritionDay
Table 59. Threat model
Table 60. Test suites
Table 61. Guardrail tests
Table 62. Environments
Table 63. Configuration variables
Table 64. Free-tier limits
Table 65. Commits on main, newest first
Table 66. Phases
Table 67. Risk register
Table 68. Decisions awaiting the owner
Table 69. Metric catalogue (migration 0003)
Table 70. Exercise catalogue (migration 0005)
Table 71. Food catalogue, per 100 g (migration 0006)

1 Introduction

This report describes the backend of **VitalLink**, a free consumer health and fitness application for the Dallas area, built as a capstone project at Kathmandu University by Group 2. It covers the analysis behind the system, its architecture, the database (with entity–relationship diagrams), the class design, every process the server runs step by step, the API, security, testing, deployment, and the current implementation status.

The backend is a single Python service (FastAPI) over a private PostgreSQL database. At the time of writing, phases 1–5b of a ten-phase build plan are complete: 18 of 35 planned tables exist, 44 HTTP operations are live, and 134 automated test cases pass against a real database.

1.1 Purpose of this document

- Explain **what** the backend does and **why** each design decision was made, so examiners and team members can evaluate it.
- Describe **how** each process works, step by step, precisely enough to re-implement or debug it.
- Record the **as-built** state (what exists in code and in the database today) separately from what is **planned**.
- Serve as the reference for the frontend team: every response shape matches `packages/api/src/types.ts`.

1.2 Scope

In scope: the API service in the repository folder `backend/`, its database schema and migrations, its tests, its container and CI pipeline, and the planned production deployment on free tiers (Supabase and Render). Out of scope: the mobile and web clients (described only where they call the API) and the native HealthKit / Health Connect sync code on the phone (phase 9).

1.3 Intended readers

Table 1. *Suggested reading order*

Reader	What to read first
Examiners / supervisors	Chapters 2 (analysis), 3 (architecture), 4 (database), 11 (status)
Backend developers	Chapters 4–7 (database, classes, processes, API) and 9 (testing)
Frontend developers	Chapter 7 (API) and section 6.1 (request lifecycle)
Operators	Chapter 10 (deployment and operations)

1.4 Definitions and acronyms

Table 2. *Definitions and acronyms*

Term	Meaning
Owner	The user the data is about
Viewer	A signed-in user reading an owner's data through a share
Share / consent / grant	One row in `consents`: owner → viewer, scopes, start, expiry, revocation
Scope	One of vitals, activity, sleep, nutrition, records; every metric belongs to exactly one

Term	Meaning
Bucket	A 5-minute window; readings are stored one row per metric per bucket per source
Metric catalogue	The closed list of 44 metrics with unit, scope, plausible range and aggregation
Idempotency key	A client-generated UUID that makes repeating a sync upload harmless
JWT / JWKS	JSON Web Token (the sign-in token) / JSON Web Key Set (the public keys that verify it)
RLS	Row-level security, a PostgreSQL feature; here enabled with no policies, i.e. deny all
UUIDv7	A time-ordered, non-guessable 128-bit identifier
ORM	Object–relational mapper (SQLAlchemy)
CI / CD	Continuous integration / continuous delivery
DFD	Data flow diagram
FTC HBNR	US Federal Trade Commission Health Breach Notification Rule

1.5 References

- Repository: `backend/README.md` and `backend/docs/01–10` (design documents), `backend/app/modules/*/README.md`.
- Frontend contract: `packages/api/src/types.ts` and `packages/api/src/client.ts`.
- VitalLink Backend System Design (reviewed design document, 30 September 2026).
- Figma file `JuPJkw5r1aAtdgRQk2UuaA` (design source of truth for every screen and response shape).

2 System analysis

A person's health data is scattered across a phone, a watch, home monitors and paper records, and there is no safe, free way to see it together or to show part of it to a doctor or relative. The backend exists to solve that problem without ever exposing the data to anyone the owner has not named.

2.1 Problem statement

- Health data lives in separate silos: HealthKit or Health Connect on the phone, a blood-pressure cuff, a glucose sensor, a smart scale, lab reports.
- Existing apps either charge a subscription, pass data to third parties for analytics or advertising, or share it through links that anyone can forward.
- A consumer who wants a doctor or family member to follow their trends has no controlled, time-limited, auditable way to do it.

2.2 Objectives

1. Collect a user's readings from the phone without duplicates, and correct them when late data arrives.
2. Turn readings, workouts and meals into dashboards, trends, totals and plain-language insights.
3. Let the owner share chosen categories of data with a named person, for a set time, and revoke at any moment.
4. Record every access decision, allowed or refused, where the owner can see it.
5. Run entirely on free hosting, with no card on file.

2.3 Stakeholders and actors

Table 3. *Actors*

Actor	Who	Goals and actions	Interface
Owner	A consumer whose data it is	Syncs, logs, reads their own data; shares and revokes; reads the access log	Mobile app, web portal
Viewer	A consumer the owner shared with: a relative, or a doctor using an ordinary account	Reads only what the share covers, until it expires or is revoked	Same apps
Phone app / health platform	Apple HealthKit, Android Health Connect	Supplies readings and workouts; the app uploads them	Ingest endpoint
Supabase Auth	Managed identity provider	Signs users in, confirms emails, issues and signs tokens	Clients and API
Scheduler	GitHub Actions cron	Triggers the nightly job (planned)	Shared secret
Project team	Group 2 developers	Build, test, deploy, operate	Repository, CI

> There is deliberately no hospital, clinic, provider or administrator actor. A doctor who views shared data signs up as an ordinary consumer and sees exactly what a family member would see.

2.4 Use case model

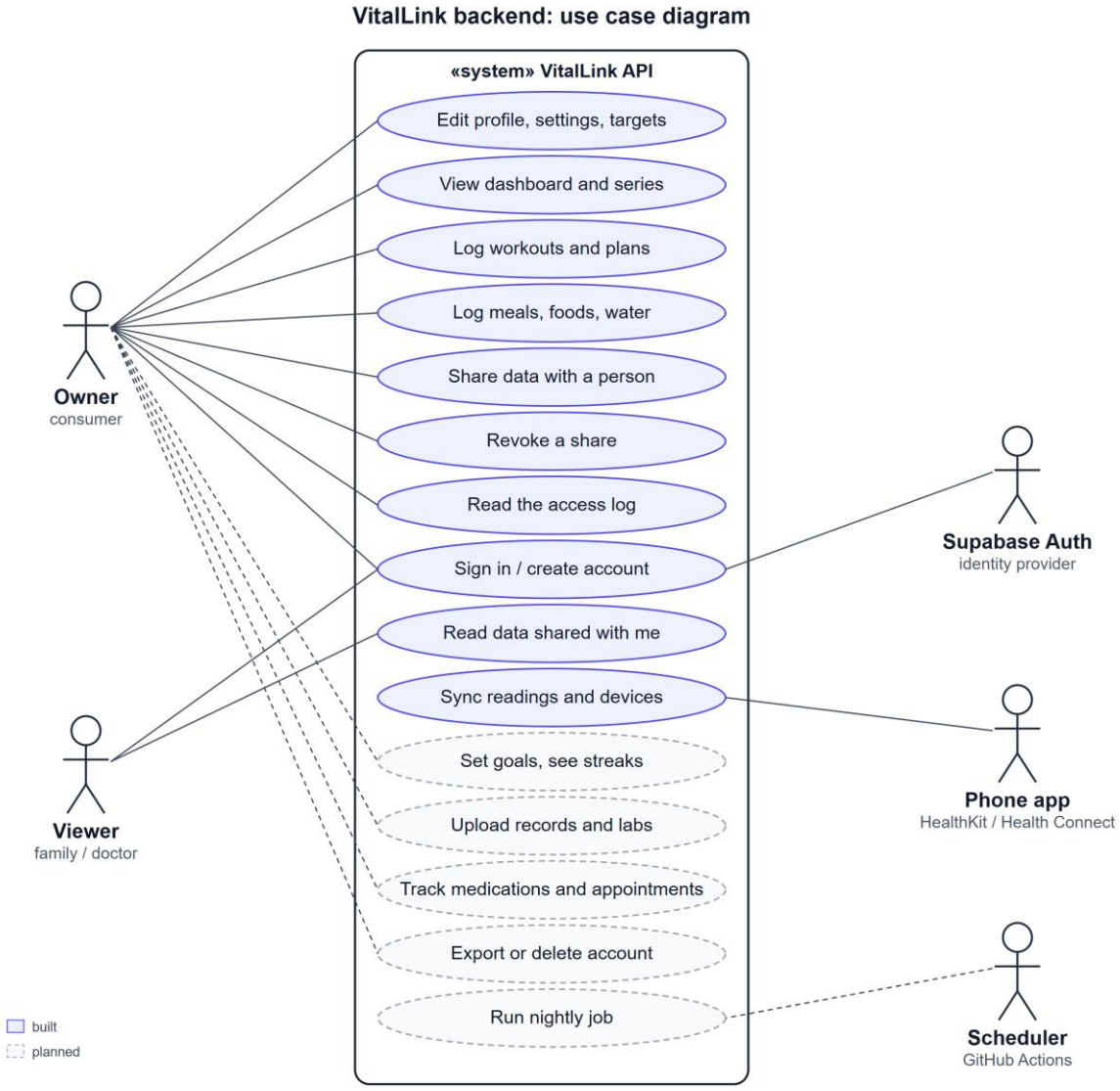


Figure 1. *Use case diagram: owners, viewers, the phone, the identity provider and the scheduler*

Solid ellipses are implemented; dashed ellipses are planned. The tables below describe each implemented use case in detail.

Table 4. *UC-01: Sign in and create account*

Field	Description
Actors	Owner or Viewer; Supabase Auth
Precondition	The user has signed up with Supabase and confirmed their email
Main flow	1. The app signs in with Supabase and receives a 1-hour access token. 2. The app calls `GET /v1/me` with the token. 3. The API verifies the token's signature, issuer, audience and expiry. 4. If no account exists for the token's subject, the API inserts one (and its profile) in a race-safe way. 5. The API returns the account: id, email, display name, time zone.

Field	Description
Alternatives	Bad or expired token → 401. Token without an email → 403 `email_required`. Email already used by another identity → 409 `email_in_use`.
Postcondition	Exactly one account and one profile exist for the user
Endpoint	`GET /v1/me`

Table 5. *UC-02: Edit profile, settings and daily targets*

Field	Description
Actor	Owner
Main flow	1. The app sends only the fields to change (`PATCH /v1/me/account` or `/me/targets`). 2. The API rejects unknown fields (no `email`, `id`, `member_since`). 3. Each value is validated (ZIP 5 digits, phone E.164, height 50–272 cm, IANA time zone, goal ranges). 4. The changes are written; the full, updated record is returned.
Alternatives	Invalid value → 422 `invalid_request` with field locations. Clearing a required setting (units, theme, language, time zone) → 422.
Postcondition	Profile updated; the database CHECK constraints guarantee the same ranges

Table 6. *UC-03: Sync readings and devices*

Field	Description
Actors	Phone app (on behalf of the Owner)
Precondition	Signed in; the phone has read merged 5-minute totals from HealthKit / Health Connect
Main flow	1. The phone sends `POST /v1/me/health/ingest` with an idempotency key, platform, devices and up to 5,000 buckets. 2. The API validates every bucket. 3. The API upserts all buckets in one statement and upserts the devices. 4. The API records the batch and returns 202 with inserted, updated and unchanged counts. 5. The phone saves its anchor only after the 202.
Alternatives	Over 5,000 buckets → 413. Same key, same payload → 202 replayed. Same key, different payload → 409. Any invalid bucket → 422 and nothing saved.
Postcondition	Each bucket stored once; late corrections overwrite earlier values

Table 7. *UC-04: View dashboard and series*

Field	Description
Actor	Owner
Main flow	1. The app requests the dashboard, a series, or latest readings. 2. The API computes them in SQL in the owner's time zone. 3. The API returns metric cards with range status, a readiness score, bucketed series or latest readings with assessments.
Endpoints	`GET /v1/me/health/dashboard`, `/series/{metric}`, `/series/blood_pressure`, `/latest`

Table 8. *UC-05: Log workouts and plans*

Field	Description
Actor	Owner (or the phone, for synced workouts)

Field	Description
Main flow	1. The app posts a workout with its sets. 2. The API checks exercise names against the catalogue and numbers the sets. 3. A synced workout with a known platform id is updated in place; otherwise a new one is created. 4. `GET /v1/me/training` returns the weekly view, sessions, records and next planned session.
Alternatives	Unknown exercise → 422. Another user's plan session → 404.

Table 9. *UC-06: Log meals, foods and water*

Field	Description
Actor	Owner
Main flow	1. The app searches foods. 2. The app posts a meal: each item is a food plus grams, or a one-off item with its own macros. 3. The API copies the macros into the meal. 4. `GET /v1/me/nutrition/day` returns totals, targets, burn estimate and meals for the day.
Alternatives	Another user's food → 404. Item with neither a food nor macros → 422.

Table 10. *UC-07: Share data with a person*

Field	Description
Actor	Owner
Main flow	1. The owner enters the person's email, a label, a kind (doctor, family, other), scopes, a purpose and an optional start and expiry. 2. The API closes any expired share to the same email. 3. The API refuses a second live share to the same person. 4. The API stores the share and writes a `share_granted` audit entry.
Alternatives	Own email → 422 `cannot_share_with_self`. Live share exists → 409 `share_exists`. Expiry in the past → 422.
Endpoint	`POST /v1/me/shares`

Table 11. *UC-08: Revoke a share*

Field	Description
Actor	Owner
Main flow	1. The owner deletes the share. 2. The API sets `revoked_at` and `revoke_reason = owner` and writes a `share_revoked` audit entry. 3. The viewer's next request is refused (nothing is cached).
Alternatives	Unknown share, or someone else's → 404

Table 12. *UC-09: Read the access log*

Field	Description
Actor	Owner
Main flow	1. The owner opens the Privacy page. 2. The API returns every read of the owner's data by someone else, newest first, with the person's label, what was accessed, the purpose or refusal reason, and the outcome.
Endpoint	`GET /v1/me/access-log?limit&offset`

Table 13. *UC-10: Read data shared with me*

Field	Description
Actor	Viewer
Precondition	The viewer's email is confirmed and an owner has shared with it
Main flow	1. `GET /v1/me/shared-with-me` lists live shares and their scopes. 2. The viewer requests data under `/v1/users/{owner}/…`. 3. The consent gate decides (eight rules), binds the grant to the viewer's account on first use and audits the decision. 4. Allowed data is returned read-only.
Alternatives	No share → 404. Scope not shared, revoked, not started, expired → 403 with the reason.

2.5 Functional requirements

Table 14. *Functional requirements*

ID	The backend must	Module	Status
FR-1	Create the account on first sign-in; keep profile, settings and daily targets	account	Built
FR-2	Accept batched 5-minute readings without duplicates; apply late corrections	health	Built
FR-3	Serve any metric as a series by hour, day, week or month in the user's time zone	health	Built
FR-4	Build the daily dashboard with a readiness score	health	Built
FR-5	Log workouts with sets; weekly load, streaks, personal records, plans	training	Built
FR-6	Log meals and water; daily totals against targets; energy balance	nutrition	Built
FR-7	Goals with progress, status and streaks	goals	Planned
FR-8	Nightly plain-language insights	goals	Planned
FR-9	Store uploaded documents behind signed links	records	Planned
FR-10	Share chosen scopes with a person for a set time; revoke at any moment	sharing	Built
FR-11	Show the owner every access, allowed or refused	sharing	Built
FR-12	Export or delete all of a user's data	account	Planned
FR-13	Lab panels with reference ranges and previous values	records	Planned
FR-14	Medications, schedules, dose log, refills	care	Planned
FR-15	Appointments the user tracks themselves (no booking)	care	Planned
FR-16	Conditions, allergies, emergency medical ID and contacts	care	Planned
FR-17	Registry of syncing devices with overdue state	health	Built
FR-18	The nine My Health tabs	health	Planned
FR-19	In-app notification feed	notifications	Planned
FR-20	Family view of people who share with me	sharing	Planned
FR-21	Health library from seeded articles	library	Planned

2.6 Non-functional requirements

Table 15. *Non-functional requirements*

Quality	Target	How it is met
Latency	95% of reads under 300 ms when warm	API and database in the same US-East region; aggregation in SQL; an index for every hot query. Measured locally: dashboard 330 ms, training view 415 ms, series about 220 ms.
Sync throughput	5,000 buckets accepted in under 2 s	One INSERT … SELECT FROM unnest(…) statement per batch
Availability	Best effort; never lose an acknowledged write	One transaction per request; idempotent ingest; the phone keeps its anchor until a 202
Privacy	No cross-user read without a live grant; every decision audited	Consent engine; audit log written in its own transaction
Security	HTTPS only; no stored passwords; no public files	Supabase Auth; private schema; signed URLs (planned)
Portability	Any PostgreSQL and any container host	All configuration through environment variables
Cost	$0	Supabase, Render and GitHub Actions free tiers
Maintainability	One language, typed, tested on every merge	Python 3.12, Pydantic, SQLAlchemy 2, ruff, pytest in CI

2.7 Feasibility study

Table 16. *Feasibility*

Dimension	Verdict	Reasoning
Technical	Feasible	Every component is mainstream (FastAPI, PostgreSQL, Supabase). Phases 1–5b already run end to end, including against the live web portal.
Economic	Feasible	$0. The free 500 MB database holds about 30 user-years of readings; the capstone needs about 15 accounts for 6 months (about 100 MB).
Operational	Feasible with care	Render sleeps after 15 minutes idle and Supabase pauses after 7 days idle. An uptime ping on /ready every 10 minutes keeps both awake.
Legal	Feasible	No provider is involved, so HIPAA does not apply; the FTC Health Breach Notification Rule does. No third-party data flows exist.
Schedule	At risk on iOS	Real HealthKit sync needs a native development build; iOS remote push needs the paid Apple Developer Program, so reminders use local notifications.

2.8 Constraints and assumptions

- **Consumers only:** no route, table or enum value for hospitals, clinics, providers or administrators. A test fails the build if one appears.
- **No blood features:** no blood type, donor or blood-bank data anywhere; blood pressure, glucose and oxygen saturation are ordinary vitals. A test checks the metric catalogue, and an exercise or food with such a name is rejected.
- **Free tiers only:** 500 MB database, 1 GB file storage, 512 MB server memory, one always-on web service.

- **Figma is the source of truth:** response shapes follow the screens; no UI is invented in the backend.
- **Assumption:** the phone uploads platform-merged 5-minute totals, not raw samples, so a watch and a phone never double-count the same steps.

2.9 Data flow diagrams

The context diagram (level 0) treats the backend as one process and shows what crosses its boundary. Level 1 opens it into six processes and six data stores.

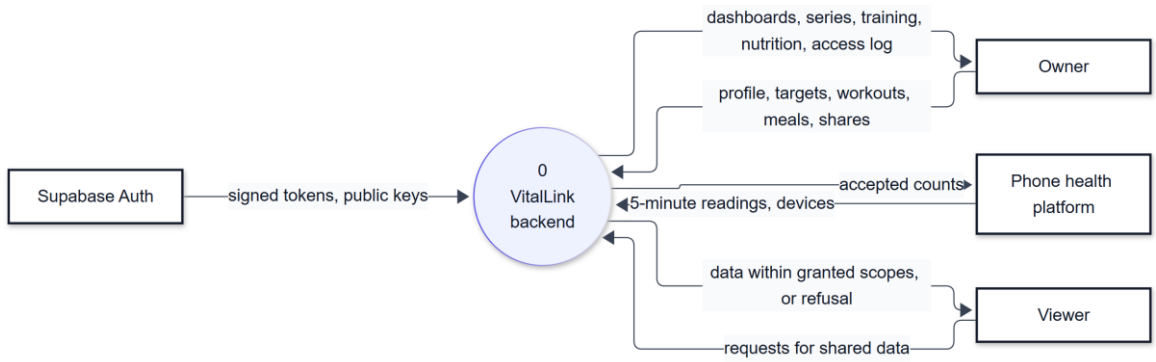


Figure 2. *Data flow diagram, level 0 (context)*

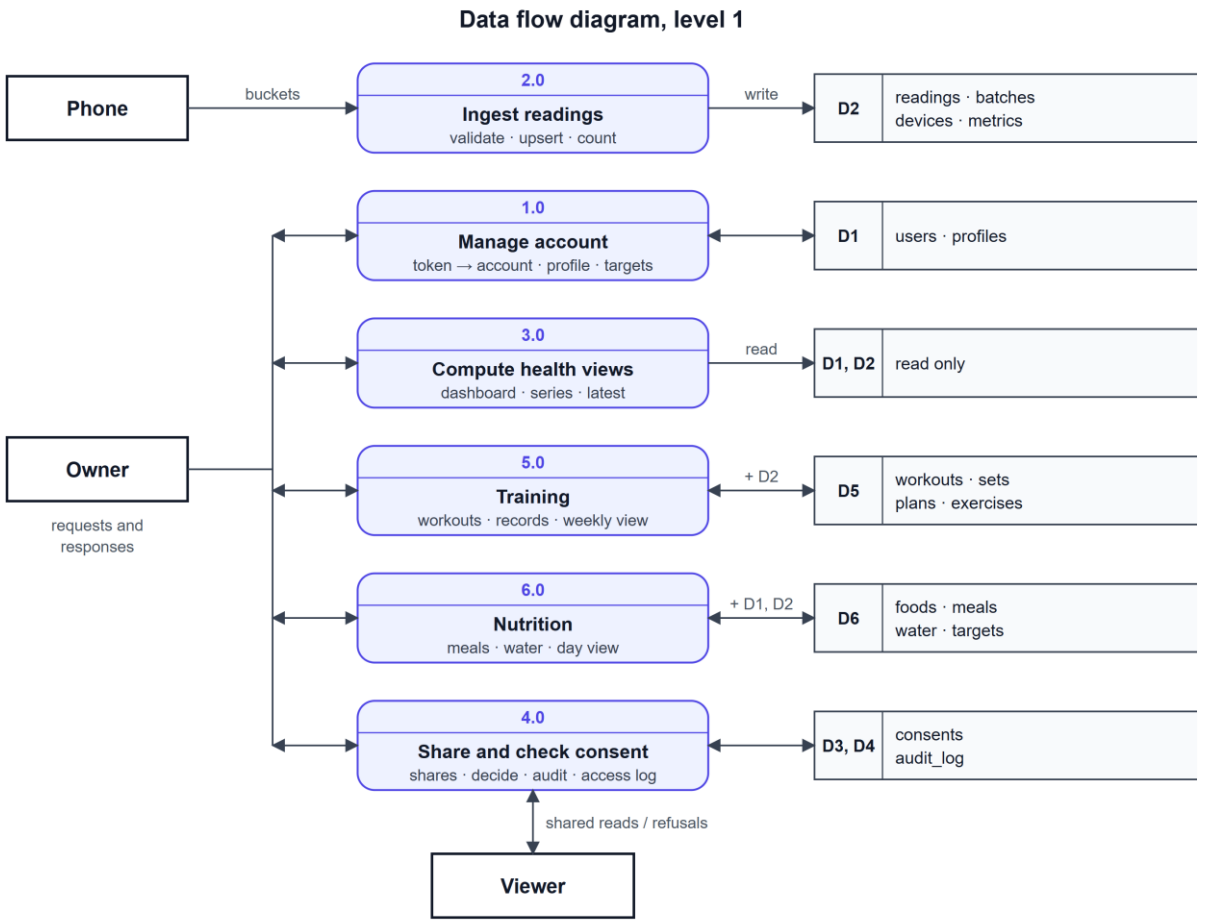


Figure 3. *Data flow diagram, level 1: six processes and six data stores*

Table 17. *Level 1 processes*

Process	Inputs	Outputs	Data stores
1.0 Manage account	Token, profile and target edits	Account, profile, targets	D1
2.0 Ingest readings	Buckets and devices from the phone	Counts (inserted / updated / unchanged)	D2
3.0 Compute health views	Readings, profile, time zone	Dashboard, series, latest readings	D1, D2
4.0 Share and check consent	Share requests, viewers' reads	Shares, allowed data or refusals, access log	D3, D4
5.0 Training	Workouts, sets, plans; active energy	Training view, records	D5, D2
6.0 Nutrition	Meals, foods, water, targets; weight	Day view, energy balance	D6, D2

3 Architecture

The backend is a **modular monolith**: one deployable FastAPI service, split inside into ten domain modules plus a shared core, talking to one PostgreSQL database that only the API can reach.

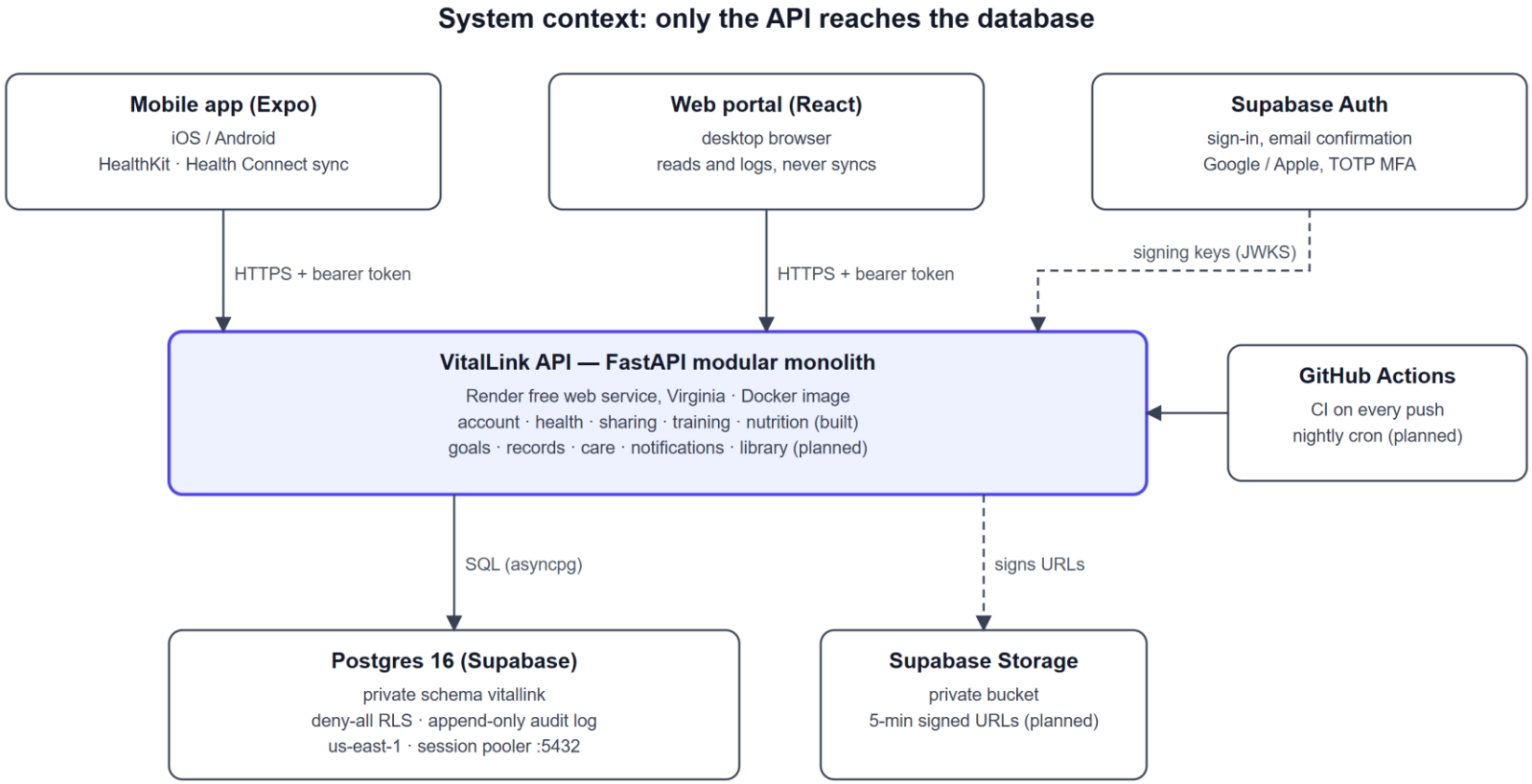


Figure 4. *System context: clients, identity provider, API, database, storage and scheduler*

Clients sign in with Supabase Auth and send the token on every call; the API verifies it against Supabase's public keys. Only the API reaches PostgreSQL. File bytes (planned) move directly between the client and Storage through short-lived links that the API signs.

3.1 Why a modular monolith

One service fits the free tier (Render offers one always-on web service), a five-person team and simple debugging. Strict internal boundaries keep the domains apart, so any module can be split out later. Microservices would add network hops, several deployments and distributed transactions, and give nothing back at this scale.

Table 18. *Architecture options considered*

Option	Gain	Cost	Verdict
Modular monolith	One deploy, one transaction per request, simple local runs	Modules scale together	Chosen
Microservices	Independent scaling	Network calls, distributed transactions, several free services (not available)	Rejected
Supabase auto-generated Data API	CRUD without code	Clients would bypass the consent engine and audit log	Rejected (and switched off)

3.2 Layers inside the service

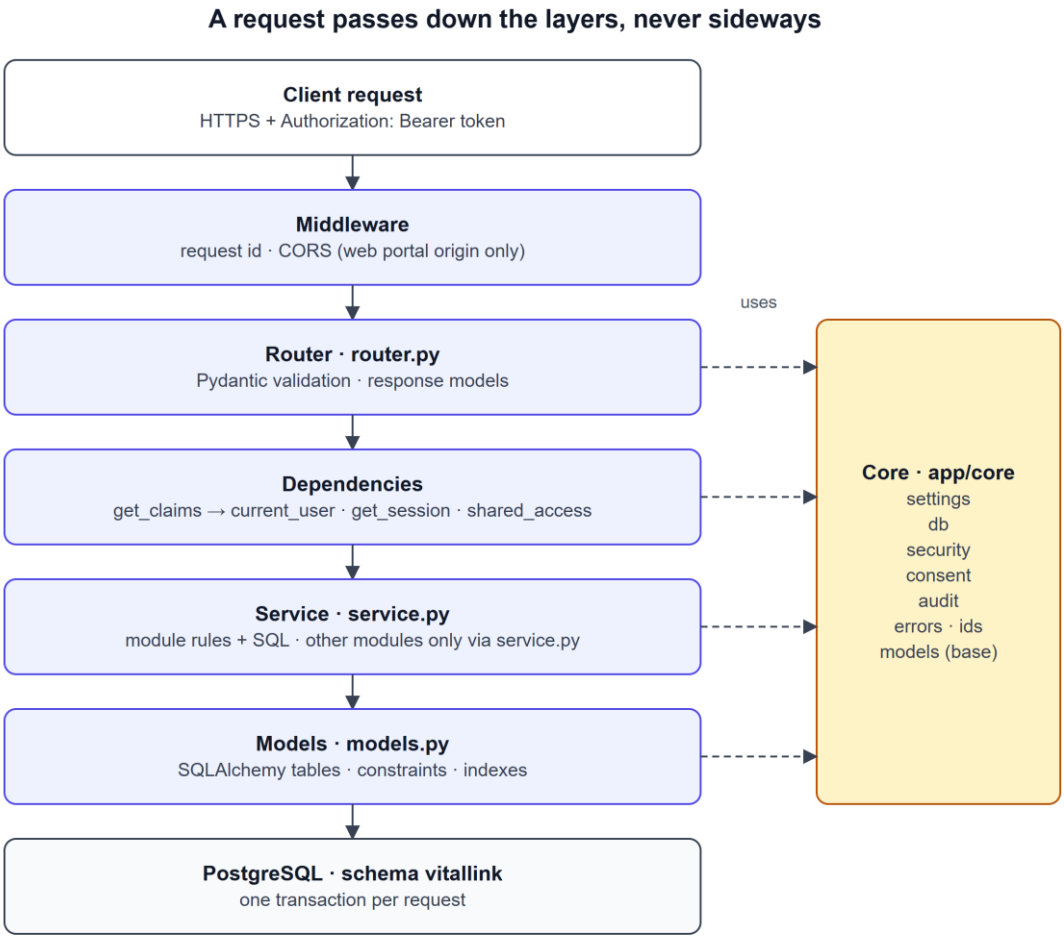


Figure 5. *Layers a request passes through*

Table 19. *Layer responsibilities*

Layer	Responsibility	Never does
Router (router.py)	Parses and validates with Pydantic, calls one service function, shapes the response	SQL, business rules
Dependencies	Resolves the caller (`get_claims` → `current_user`), opens the transaction (`get_session`), opens the consent gate (`shared_access`)	Load module data
Service (service.py)	Module rules and queries; calls other modules only through their service.py	Touch another module's tables directly
Models (models.py)	SQLAlchemy mappings with constraints and indexes	Decide anything
Core (app/core)	Settings, database, security, consent, audit, errors, ids, model base	Import from a module

3.3 Modules

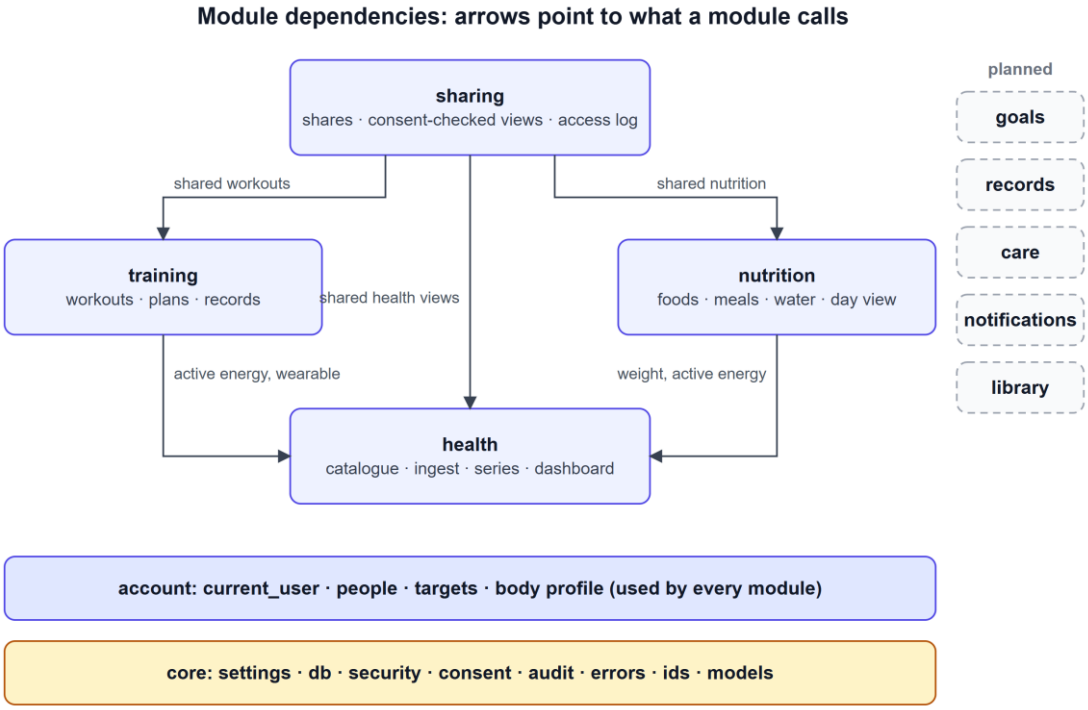


Figure 6. *Module dependencies (arrows point to what a module calls)*

Table 20. *Modules and table ownership*

Module	Responsibility	Owns tables	Status
account	Account from token, profile, settings, targets	users, profiles	Built
health	Metric catalogue, ingest, series, latest, dashboard, readiness, devices	metrics, health*readings, ingest*batches, devices	Built
sharing	Shares, consent-checked views, shared-with-me, access log	consents (+ audit_log via core)	Built
training	Workouts, sets, plans, weekly view, records	exercises, workouts, exercise*sets, training*plans, plan_sessions	Built
nutrition	Foods, meals, water, targets, day view	foods, meal*entries, meal*items, hydration*logs, nutrition*targets	Built
goals	Goals, progress, streaks, insights	goals, goal_progress, insights	Planned
records	Documents, signed upload/download, lab panels	health*records, lab*panels, lab_results	Planned
care	Medications, doses, appointments, conditions, emergency ID	7 tables	Planned
notifications	In-app feed	notifications	Planned
library	Seeded health articles	article*categories, articles, library*features	Planned

3.4 Shared core

Table 21. *Core components*

File	Responsibility
settings.py	Reads every configuration value from environment variables (pydantic-settings); nothing environment-specific in code
db.py	One async engine (pool 5 + 5 overflow; no pooling in tests); `get_session` opens one transaction per request, commits on success and rolls back on any error; `ping()` for /ready
security.py	Verifies bearer tokens: Supabase JWKS (ES256/RS256, keys cached 10 min) in production; HS256 dev tokens only when ENV is local or test; `mint_dev_token` for local use
consent.py	`decide()` (pure), `Grant`, `Viewer`, `Decision`, and `ConsentGate` (loads grants, binds, audits, raises)
audit.py	Audit table mapping, enums, and `write_audit()`, the only writer, on its own connection
errors.py	`ApiError` and handlers that turn every failure into one JSON envelope; validation errors never echo values
ids.py	UUIDv7 generator (48-bit millisecond time + 74 random bits)
models.py	Declarative base pinned to the `vitallink` schema; constraint naming convention; Postgres enum helper

3.5 Technology stack

Table 22. *Technology stack*

Concern	Choice	Reason
Language and framework	Python 3.12, FastAPI, Pydantic v2	One language for API and analysis; OpenAPI generated from code; typed requests and responses
Database access	SQLAlchemy 2 (async) + asyncpg	Bound parameters only; async fits FastAPI
Migrations	Alembic	Versioned, reviewed schema changes; a test checks models and migrations agree
Database	PostgreSQL 16	Constraints, enums, partial indexes, arrays; Supabase free tier in production, Docker locally
Auth	Supabase Auth, verified by the API (PyJWT)	No passwords stored by us; email confirmation, Google, Apple, TOTP MFA
Files	Supabase Storage, private bucket (planned)	5-minute signed links; bytes never pass through the API
Hosting	Render free web service (Virginia), Docker	Same region as the database, about 30 ms from Dallas
Scheduling	GitHub Actions cron (planned)	Free; a failed run turns red
Quality	pytest + httpx on real PostgreSQL, ruff	The guarantees that matter live in the database

3.6 Source layout

```
backend/
  app/
    main.py          app: middleware, error handlers, /health, /ready, /v1 routers
    devtoken.py      prints a local dev token: python -m app.devtoken <email>
    models.py        imports every module's models (Alembic, drift check)
    core/            settings, db, security, consent, audit, errors, ids, models
```

```
  modules/
    account/       models.py  schemas.py  service.py  router.py
    health/        catalogue.py (44 metrics) + the four files
    sharing/       consents; /me/shares, /me/access-log, /users/{id}/... views
    training/      catalogue.py (30 exercises) + the four files
    nutrition/     catalogue.py (30 foods) + the four files
    goals/ records/ care/ notifications/ library/    README only (planned)
migrations/        env.py + versions/0001 ... 0006
seeds/demo.py      90-day demo account
tests/             conftest.py + 9 test files
Dockerfile  start.sh  docker-compose.yml  alembic.ini  pyproject.toml
docs/              design documents 01-10 and this report
```

4 Database design

The database has **18 tables today** (migrations 0001–0006) and 35 when the plan is complete, all in the private `vitallink` schema. The schema itself enforces the rules: a bad value is refused by PostgreSQL, not only by the API.

4.1 Design rules

Table 23. *Schema-wide rules*

Rule	How it is enforced
No direct access	Tables live in `vitallink`, which Supabase's Data API does not expose; row-level security is enabled on every table with no policies (deny all); all privileges revoked from PUBLIC, anon and authenticated
Ownership	Every user-owned table has `user_id → users.id ON DELETE CASCADE` and an index starting with `user_id`
Identifiers	UUIDv7 for every id a client sees; bigint identity for internal rows; uuid5 of the name for seeded foods (stable across databases)
NOT NULL by default	A nullable column has a stated meaning, e.g. `email_confirmed_at` null = not confirmed
Closed sets	18 PostgreSQL enums; the metric and exercise catalogues are lookup tables
Time	`timestamptz` in UTC; calendar facts (birth date) are `date`; days are grouped in the user's IANA time zone
Units	SI in the database (m, kg, ml, s); clients convert per the user's unit setting
No soft delete	Hard deletes cascade from users; rows that stay but go inactive use a status enum
Migrations only	Alembic; CI applies, reverses and re-applies them; a test fails if models and migrations disagree

4.2 Entity–relationship diagrams

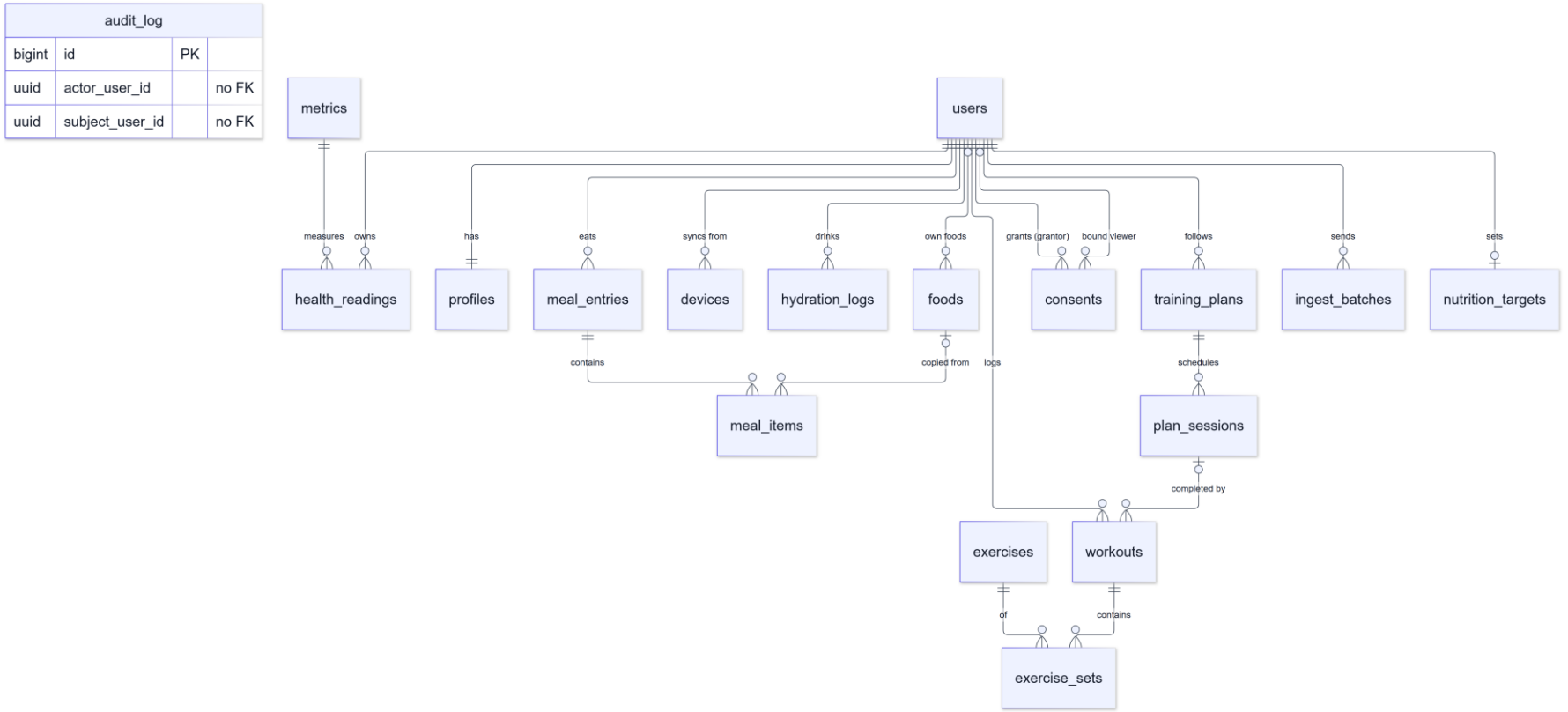


Figure 7. *ER diagram, as built: all 18 tables and their relationships*

`audit_log` is deliberately unconnected. It stores user and consent ids without foreign keys, so entries outlive deleted accounts and no cascade can ever rewrite them.

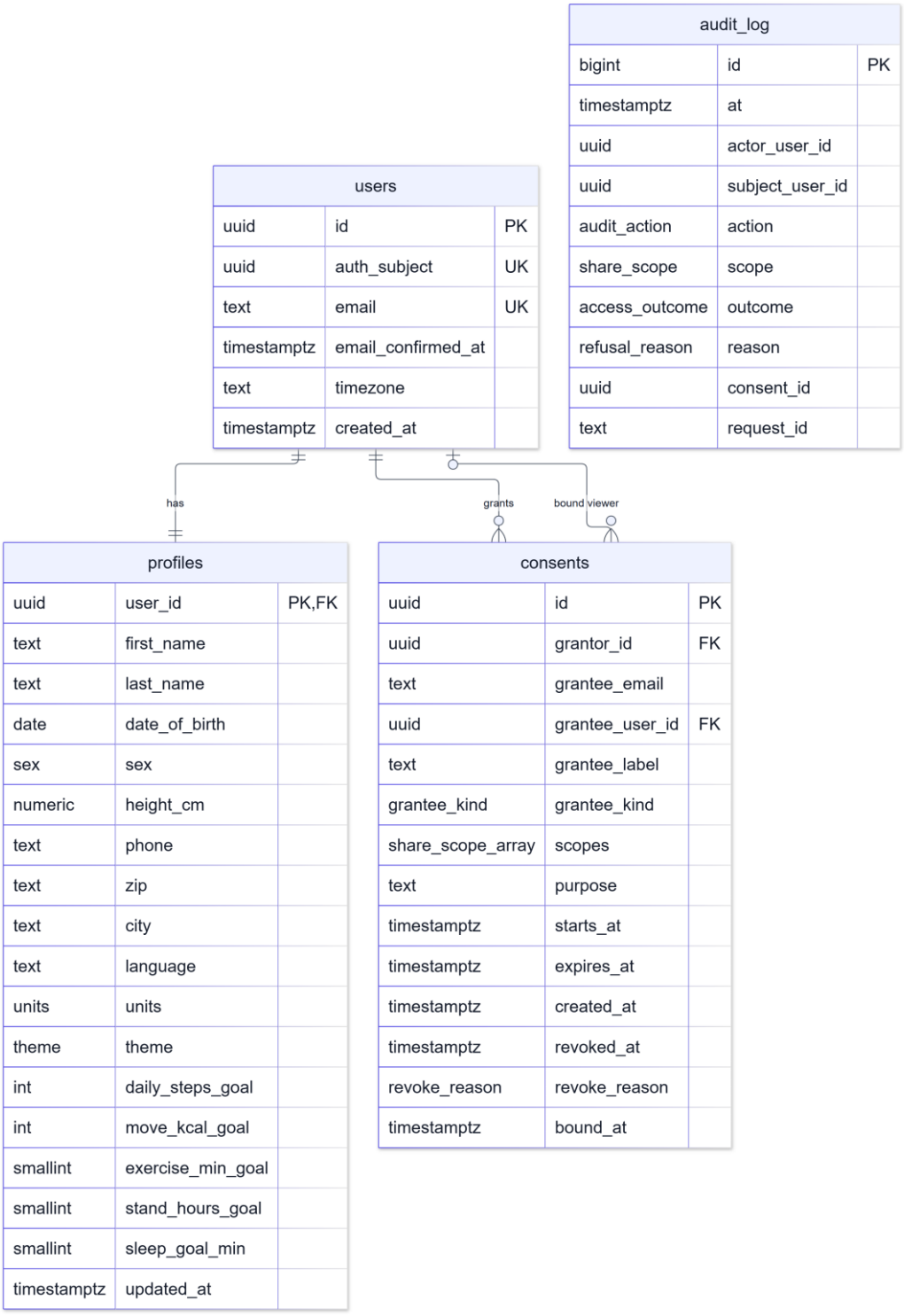


Figure 8. *ER detail: accounts, sharing and the audit log*

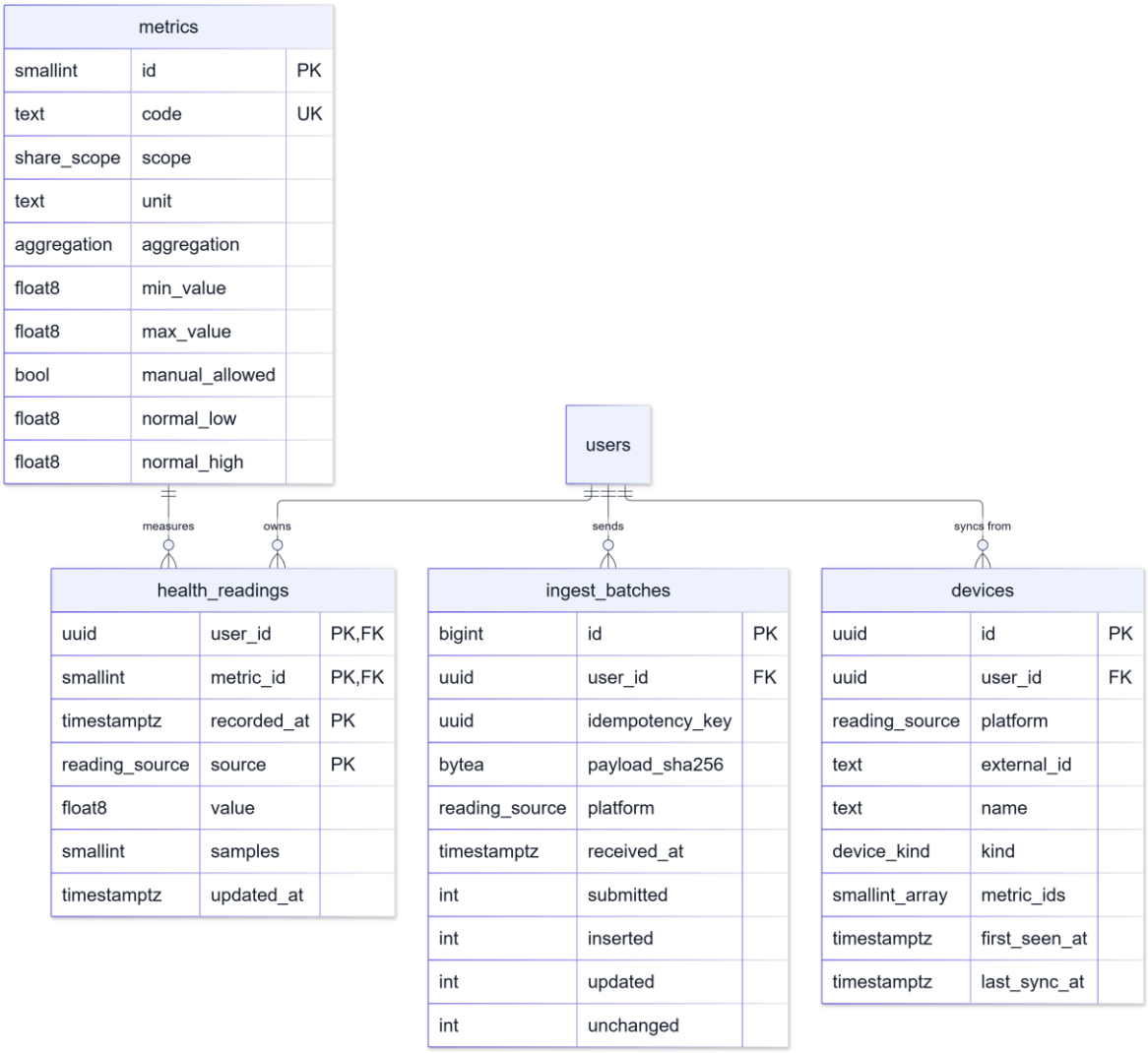


Figure 9. *ER detail: health readings, ingest batches, devices and the metric catalogue*

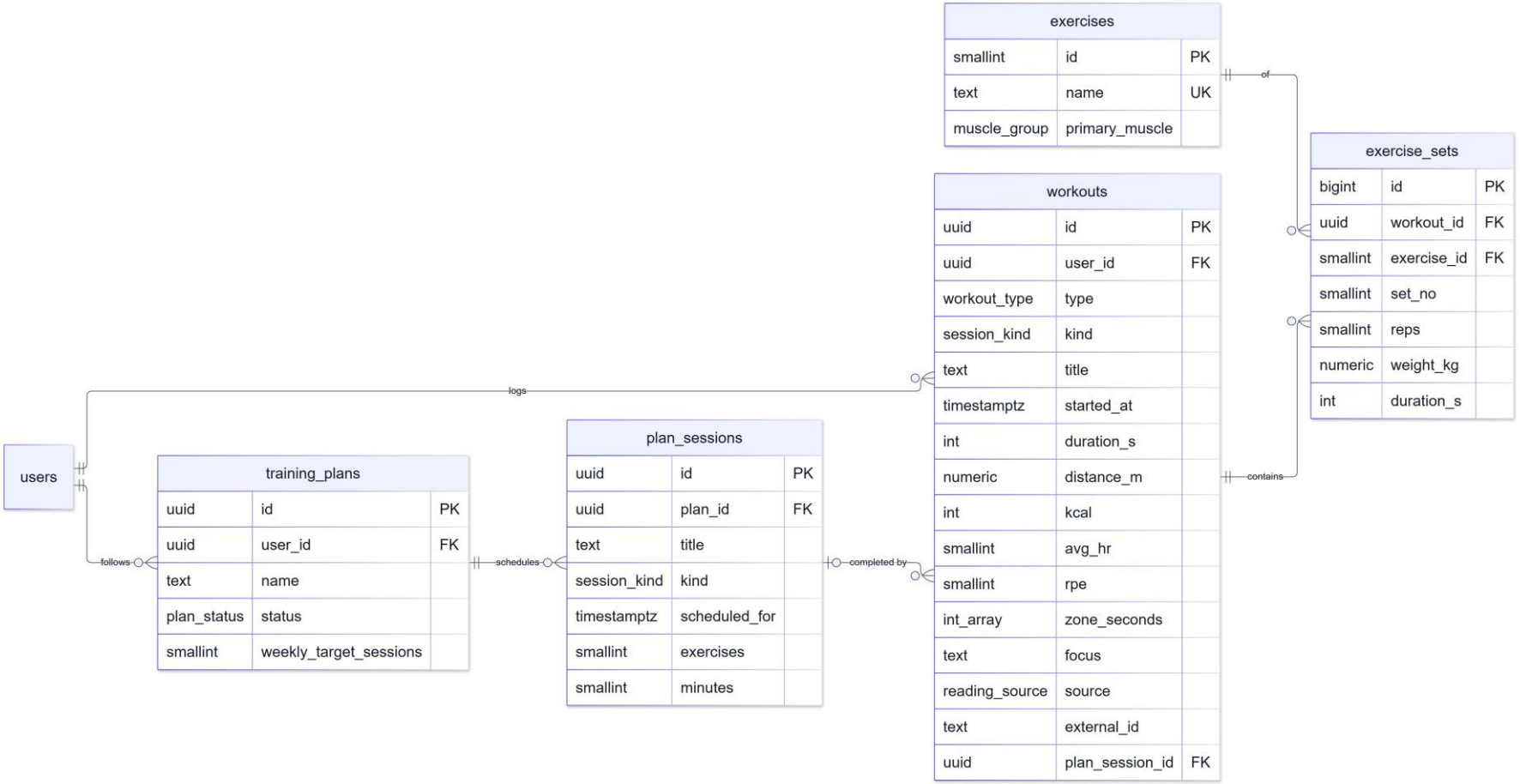


Figure 10. *ER detail: training*

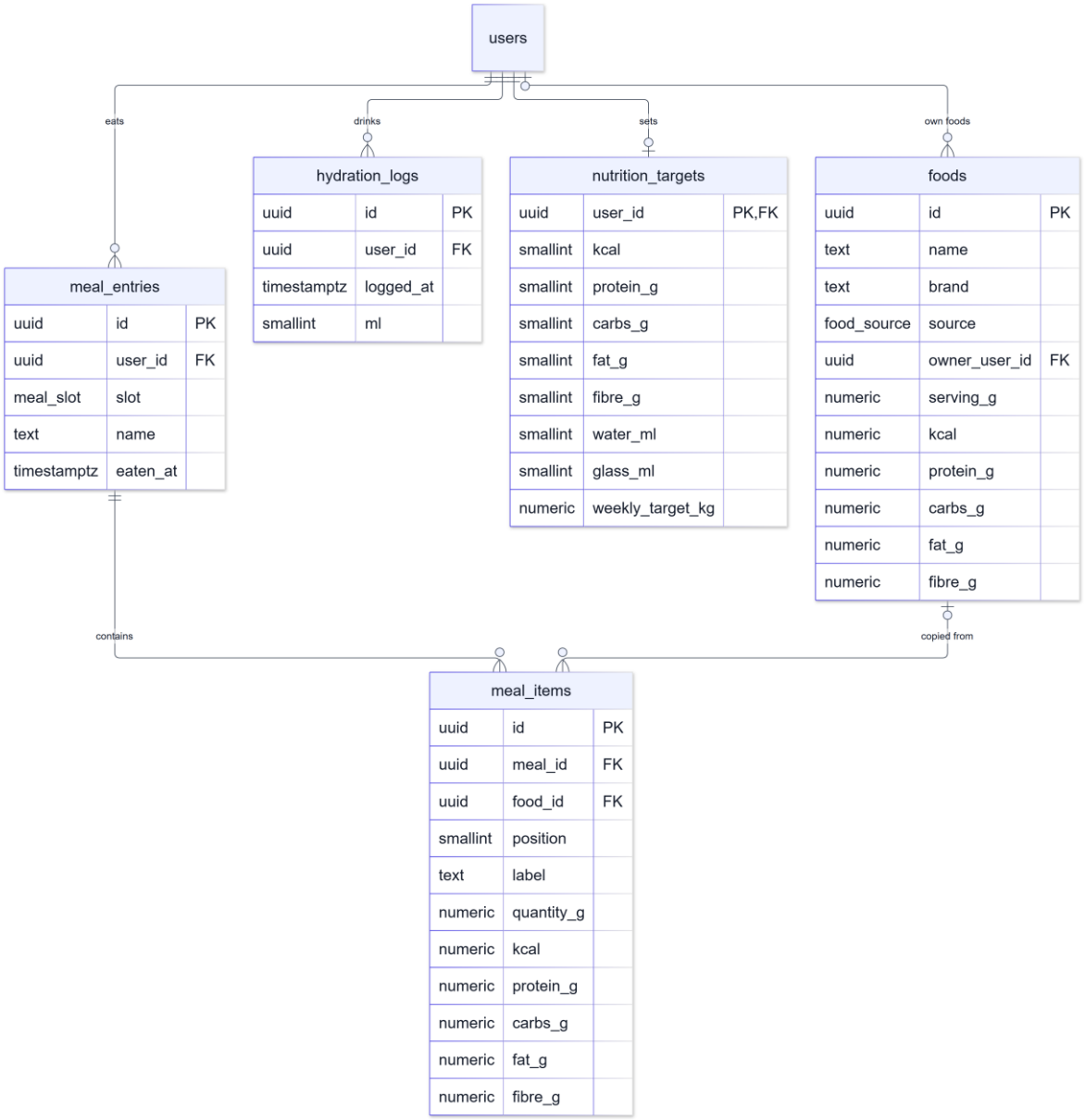


Figure 11. *ER detail: nutrition*

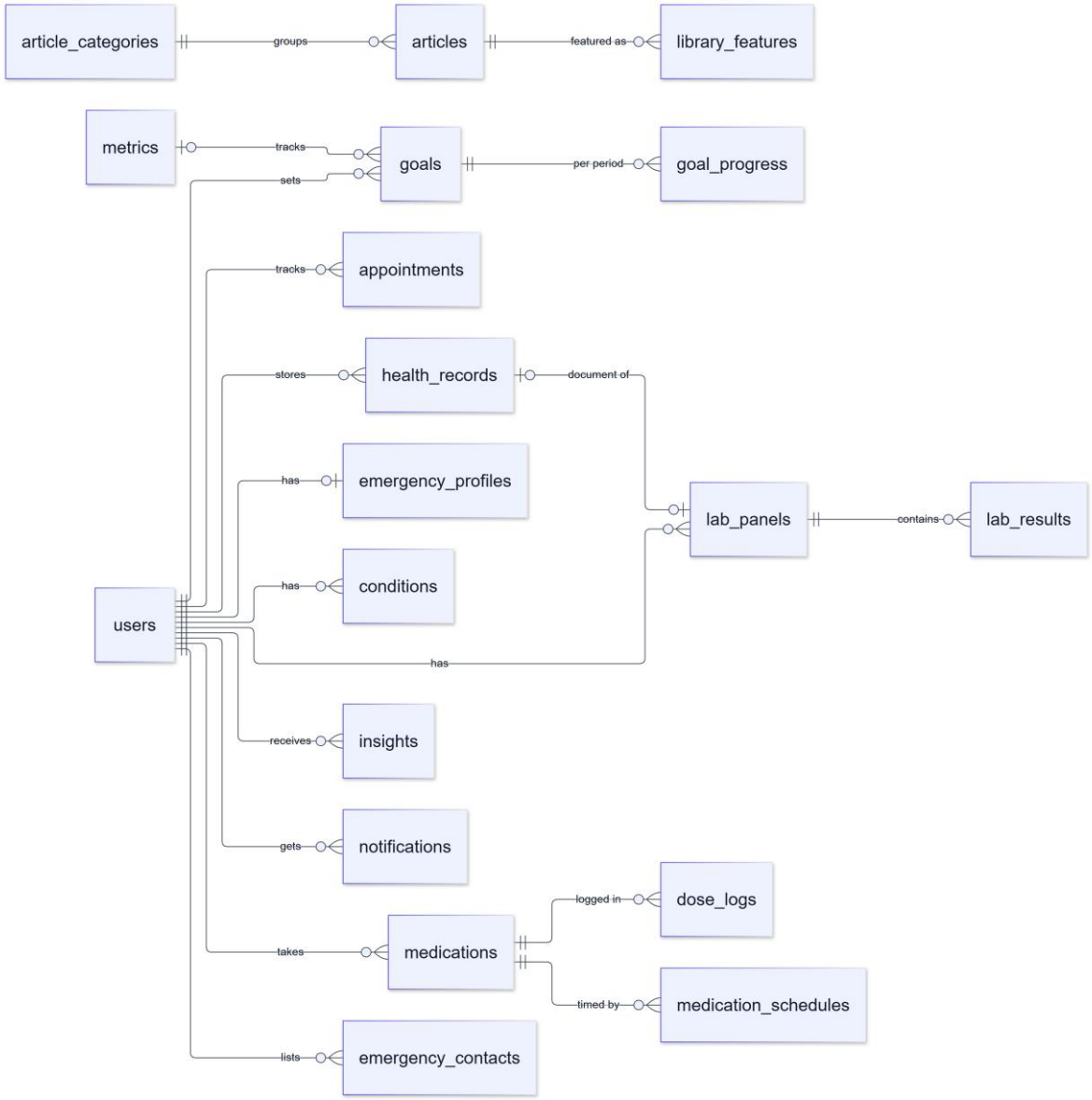


Figure 12. *ER diagram, planned: the 17 tables of phases 5c–8*

4.3 Relationship summary

Table 24. *Relationships*

Parent	Child	Cardinality	On delete	Meaning
users	profiles	1 : 1	CASCADE	Each account has exactly one profile
users	consents (grantor)	1 : N	CASCADE	An owner grants many shares
users	consents (grantee*user*id)	0..1 : N	RESTRICT	A share binds to its viewer on first read; deletion must revoke first
users	health*readings, ingest*batches, devices	1 : N	CASCADE	Readings and sync records belong to their owner
metrics	health_readings	1 : N	RESTRICT	Every reading is of one catalogue metric
users	workouts, training_plans	1 : N	CASCADE	
workouts	exercise_sets	1 : N	CASCADE	Sets belong to a workout

Parent	Child	Cardinality	On delete	Meaning
exercises	exercise_sets	1 : N	RESTRICT	Catalogue rows are never deleted
training_plans	plan_sessions	1 : N	CASCADE	
plan_sessions	workouts	0..1 : N	SET NULL	A workout may complete a planned session
users	meal*entries, hydration*logs	1 : N	CASCADE	
meal_entries	meal_items	1 : N	CASCADE	
foods	meal_items	0..1 : N	SET NULL	Items keep their copied macros if the food goes
users	foods (owner*user*id)	0..1 : N	CASCADE	Users' own foods
users	nutrition_targets	1 : 0..1	CASCADE	Created with defaults on first read

4.4 Data dictionary

The tables below are generated from the live database (information*schema and pg*catalog), so they show exactly what migration 0006 produces.

4.4.1 users

One row per account. Created on the first authenticated request.

Column	Type	Null	Default
id	uuid	no	
auth_subject	uuid	no	
email	text	no	
email*confirmed*at	timestamptz	yes	
timezone	text	no	America/Chicago
created_at	timestamptz	no	now()

Constraint / index	Kind	Definition
users*email*lowercase	b'c'	CHECK ((email = lower(email)))
users_pkey	b'p'	PRIMARY KEY (id)
users*auth*subject_key	b'u'	UNIQUE (auth_subject)
users*email*key	b'u'	UNIQUE (email)

4.4.2 profiles

Profile, settings and daily targets; 1:1 with users.

Column	Type	Null	Default
user_id	uuid	no	
first_name	text	yes	
last_name	text	yes	

Column	Type	Null	Default
date*of*birth	date	yes	
sex	sex	yes	
height_cm	numeric(4,1)	yes	
phone	text	yes	
zip	text	yes	
city	text	yes	
language	text	no	en-US
units	units	no	imperial
theme	theme	no	system
daily*steps*goal	integer	no	10000
move*kcal*goal	integer	no	500
exercise*min*goal	smallint	no	30
stand*hours*goal	smallint	no	12
sleep*goal*min	smallint	no	480
updated_at	timestamptz	no	now()

Constraint / index	Kind	Definition
profiles*dob*range	b'c'	CHECK ((date*of*birth >= '1900-01-01'::date))
profiles*exercise*goal_range	b'c'	CHECK (((exercise*min*goal >= 5) AND (exercise*min*goal <= 600)))
profiles*height*trange	b'c'	CHECK (((height*cm >= (50)::numeric) AND (height*cm <= (272)::numeric)))
profiles*move*goal_range	b'c'	CHECK (((move*kcal*goal >= 50) AND (move*kcal*goal <= 5000)))
profiles*phone*e164	b'c'	CHECK ((phone ~ '^\+[1-9][0-9]{6,14}$'::text))
profiles*sleep*goal_range	b'c'	CHECK (((sleep*goal*min >= 180) AND (sleep*goal*min <= 900)))
profiles*stand*goal_range	b'c'	CHECK (((stand*hours*goal >= 1) AND (stand*hours*goal <= 24)))
profiles*steps*goal_range	b'c'	CHECK (((daily*steps*goal >= 1000) AND (daily*steps*goal <= 100000)))
profiles*zip*format	b'c'	CHECK ((zip ~ '^[0-9]{5}$'::text))
profiles*user*id_fkey	b'f'	FOREIGN KEY (user_id) REFERENCES users(id) ON DELETE CASCADE
profiles_pkey	b'p'	PRIMARY KEY (user_id)

4.4.3 consents

One share from an owner to a person, with scopes, validity window and binding.

Column	Type	Null	Default
id	uuid	no	
grantor_id	uuid	no	
grantee_email	text	no	
grantee*user*id	uuid	yes	
grantee_label	text	no	

Column	Type	Null	Default
grantee_kind	grantee_kind	no	
scopes	share_scope[]	no	
purpose	text	no	
starts_at	timestamptz	no	now()
expires_at	timestamptz	yes	
created_at	timestamptz	no	now()
revoked_at	timestamptz	yes	
revoke_reason	revoke_reason	yes	
bound_at	timestamptz	yes	

Constraint / index	Kind	Definition
consents*binding*has_time	b'c'	CHECK (((grantee*user*id IS NULL) OR (bound_at IS NOT NULL)))
consents*binding*kept	b'c'	CHECK (((bound*at IS NULL) OR (grantee*user*id IS NOT NULL) OR (revoked*at IS NOT NULL)))
consents*email*lowercase	b'c'	CHECK ((grantee*email = lower(grantee*email)))
consents*expiry*after_start	b'c'	CHECK (((expires*at IS NULL) OR (expires*at > starts_at)))
consents*label*length	b'c'	CHECK (((char*length(grantee*label) >= 1) AND (char*length(grantee*label) <= 80)))
consents*not*self	b'c'	CHECK (((grantee*user*id IS NULL) OR (grantee*user*id <> grantor_id)))
consents*purpose*length	b'c'	CHECK ((char_length(purpose) <= 200))
consents*revocation*has_reason	b'c'	CHECK (((revoked*at IS NULL) = (revoke*reason IS NULL)))
consents*scopes*present	b'c'	CHECK ((cardinality(scopes) > 0))
consents*grantee*user*id*fkey	b'f'	FOREIGN KEY (grantee*user*id) REFERENCES users(id) ON DELETE RESTRICT
consents*grantor*id_fkey	b'f'	FOREIGN KEY (grantor_id) REFERENCES users(id) ON DELETE CASCADE
consents_pkey	b'p'	PRIMARY KEY (id)
consents*grantee*user*id*idx	index	(grantee*user*id)
consents*grantor*id_idx	index	(grantor_id)
consents*one*live*per*grantee	index	UNIQUE (grantor*id, grantee*email) WHERE (revoked_at IS NULL)
consents*unbound*email_idx	index	(grantee*email) WHERE (grantee*user_id IS NULL)

4.4.4 audit_log

Every access decision and share event. Append-only: triggers block UPDATE, DELETE and TRUNCATE.

Column	Type	Null	Default
id	bigint	no	
at	timestamptz	no	now()
actor*user*id	uuid	yes	
subject*user*id	uuid	no	
action	audit_action	no	

Column	Type	Null	Default
scope	share_scope	yes	
outcome	access_outcome	no	
reason	refusal_reason	yes	
consent_id	uuid	yes	
request_id	text	yes	

Constraint / index	Kind	Definition
audit*read*has_actor	b'c'	CHECK (((action <> 'read'::audit*action) OR (actor*user_id IS NOT NULL)))
audit*read*has_scope	b'c'	CHECK (((action <> 'read'::audit_action) OR (scope IS NOT NULL)))
audit*reason*only*when*blocked	b'c'	CHECK (((outcome = 'blocked'::access_outcome) = (reason IS NOT NULL)))
audit*log*pkey	b'p'	PRIMARY KEY (id)
audit*log*actor_at	index	(actor*user*id, at DESC)
audit*log*subject_at	index	(subject*user*id, at DESC)
audit*log*no*update*delete	trigger	BEFORE DELETE, UPDATE: raises "append-only"

4.4.5 metrics

The closed catalogue of 44 metrics, seeded by migration 0003.

Column	Type	Null	Default
id	smallint	no	
code	text	no	
scope	share_scope	no	
unit	text	no	
aggregation	aggregation	no	
min_value	double precision	no	
max_value	double precision	no	
manual_allowed	boolean	no	
normal_low	double precision	yes	
normal_high	double precision	yes	

Constraint / index	Kind	Definition
metrics*normal*order	b'c'	CHECK (((normal*low IS NULL) OR (normal*high IS NULL) OR (normal*low < normal*high)))
metrics*range*order	b'c'	CHECK ((min*value < max*value))
metrics_pkey	b'p'	PRIMARY KEY (id)
metrics*code*key	b'u'	UNIQUE (code)

4.4.6 health_readings

One value per metric per 5-minute bucket per source. The primary key is the de-duplication key.

Column	Type	Null	Default
user_id	uuid	no	
metric_id	smallint	no	
recorded_at	timestamptz	no	
source	reading_source	no	
value	double precision	no	
samples	smallint	no	
updated_at	timestamptz	no	now()

Constraint / index	Kind	Definition
health*readings*bucket_aligned	b'c'	CHECK ((((EXTRACT(epoch FROM recorded_at))::bigint % (300)::bigint) = 0))
health*readings*samples_positive	b'c'	CHECK ((samples > 0))
health*readings*metric*id*fkey	b'f'	FOREIGN KEY (metric_id) REFERENCES metrics(id) ON DELETE RESTRICT
health*readings*user*id*fkey	b'f'	FOREIGN KEY (user_id) REFERENCES users(id) ON DELETE CASCADE
health*readings*pkey	b'p'	PRIMARY KEY (user*id, metric*id, recorded_at, source)

4.4.7 ingest_batches

Idempotency record of each sync upload.

Column	Type	Null	Default
id	bigint	no	
user_id	uuid	no	
idempotency_key	uuid	no	
payload_sha256	bytea	no	
platform	reading_source	no	
received_at	timestamptz	no	now()
submitted	integer	no	
inserted	integer	no	
updated	integer	no	
unchanged	integer	no	

Constraint / index	Kind	Definition
ingest*batches*counts*add*up	b'c'	CHECK ((((inserted + updated) + unchanged) = submitted))
ingest*batches*hash_length	b'c'	CHECK ((octet*length(payload*sha256) = 32))
ingest*batches*submitted_range	b'c'	CHECK (((submitted >= 1) AND (submitted <= 5000)))
ingest*batches*user*id*fkey	b'f'	FOREIGN KEY (user_id) REFERENCES users(id) ON DELETE CASCADE

Constraint / index	Kind	Definition
ingest*batches*pkey	b'p'	PRIMARY KEY (id)
ingest*batches*user*key*key	b'u'	UNIQUE (user*id, idempotency*key)
ingest*batches*received*at*idx	index	(received_at)

4.4.8 devices

Devices that contributed readings, with last sync time.

Column	Type	Null	Default
id	uuid	no	
user_id	uuid	no	
platform	reading_source	no	
external_id	text	no	
name	text	no	
kind	device_kind	no	
metric_ids	int2[]	no	
first*seen*at	timestamptz	no	now()
last*sync*at	timestamptz	no	now()

Constraint / index	Kind	Definition
devices*user*id_fkey	b'f'	FOREIGN KEY (user_id) REFERENCES users(id) ON DELETE CASCADE
devices_pkey	b'p'	PRIMARY KEY (id)
devices*user*platform*external*key	b'u'	UNIQUE (user*id, platform, external*id)

4.4.9 exercises

Catalogue of 30 exercises with primary muscle group, seeded by migration 0005.

Column	Type	Null	Default
id	smallint	no	
name	text	no	
primary_muscle	muscle_group	no	

Constraint / index	Kind	Definition
exercises_pkey	b'p'	PRIMARY KEY (id)
exercises*name*key	b'u'	UNIQUE (name)

4.4.10 workouts

One training session, manual or synced.

Column	Type	Null	Default
id	uuid	no	
user_id	uuid	no	
type	workout_type	no	
kind	session_kind	no	
title	text	no	
started_at	timestamptz	no	
duration_s	integer	no	
distance_m	numeric(9,1)	yes	
kcal	integer	yes	
avg_hr	smallint	yes	
rpe	smallint	yes	
zone_seconds	int4[]	yes	
focus	text	yes	
source	reading_source	no	
external_id	text	yes	
plan*session*id	uuid	yes	
created_at	timestamptz	no	now()

Constraint / index	Kind	Definition
workouts*avg*hr_range	b'c'	CHECK (((av*ghr >= 25) AND (avg*hr <= 250)))
workouts*distance*nonnegative	b'c'	CHECK ((distance_m >= (0)::numeric))
workouts*duration*range	b'c'	CHECK (((duration*s >= 1) AND (duration*s <= 86400)))
workouts*external*id*matches*source	b'c'	CHECK (((source = 'manual'::reading*source) = (external*id IS NULL)))
workouts*kcal*range	b'c'	CHECK (((kcal >= 0) AND (kcal <= 20000)))
workouts*rpe*range	b'c'	CHECK (((rpe >= 1) AND (rpe <= 10)))
workouts*title*length	b'c'	CHECK (((char*length(title) >= 1) AND (char*length(title) <= 80)))
workouts*zones*shape	b'c'	CHECK (((zone*seconds IS NULL) OR ((cardinality(zone*seconds) = 5) AND (0 <= ALL (zone_seconds)))))
workouts*plan*session*id*fkey	b'f'	FOREIGN KEY (plan*session*id) REFERENCES plan_sessions(id) ON DELETE SET NULL
workouts*user*id_fkey	b'f'	FOREIGN KEY (user_id) REFERENCES users(id) ON DELETE CASCADE
workouts_pkey	b'p'	PRIMARY KEY (id)
workouts*plan*session*id*idx	index	(plan*session*id)
workouts*user*source*external*key	index	UNIQUE (user*id, source, external*id) WHERE (external_id IS NOT NULL)
workouts*user*started_idx	index	(user*id, started*at DESC)

4.4.11 exercise_sets

One set within a workout.

Column	Type	Null	Default
id	bigint	no	
workout_id	uuid	no	
exercise_id	smallint	no	
set_no	smallint	no	
reps	smallint	yes	
weight_kg	numeric(6,2)	yes	
duration_s	integer	yes	

Constraint / index	Kind	Definition
exercise*sets*duration_positive	b'c'	CHECK ((duration_s > 0))
exercise*sets*reps*or*duration	b'c'	CHECK (((reps IS NOT NULL) OR (duration_s IS NOT NULL)))
exercise*sets*reps_range	b'c'	CHECK (((reps >= 1) AND (reps <= 1000)))
exercise*sets*set*no*positive	b'c'	CHECK ((set_no > 0))
exercise*sets*weight_range	b'c'	CHECK (((weight*kg >= (0)::numeric) AND (weight*kg <= (1000)::numeric)))
exercise*sets*exercise*id*fkey	b'f'	FOREIGN KEY (exercise_id) REFERENCES exercises(id) ON DELETE RESTRICT
exercise*sets*workout*id*fkey	b'f'	FOREIGN KEY (workout_id) REFERENCES workouts(id) ON DELETE CASCADE
exercise*sets*pkey	b'p'	PRIMARY KEY (id)
exercise*sets*workout*exercise*set_key	b'u'	UNIQUE (workout*id, exercise*id, set_no)
exercise*sets*exercise*id*idx	index	(exercise_id)

4.4.12 training_plans

A weekly training plan; at most one active per user.

Column	Type	Null	Default
id	uuid	no	
user_id	uuid	no	
name	text	no	
status	plan_status	no	
weekly*target*sessions	smallint	no	
created_at	timestamptz	no	now()

Constraint / index	Kind	Definition
training*plans*name_length	b'c'	CHECK (((char*length(name) >= 1) AND (char*length(name) <= 80)))
training*plans*target_range	b'c'	CHECK (((weekly*target*sessions >= 1) AND (weekly*target*sessions <= 14)))
training*plans*user*id*fkey	b'f'	FOREIGN KEY (user_id) REFERENCES users(id) ON DELETE CASCADE
training*plans*pkey	b'p'	PRIMARY KEY (id)
training*plans*one_active	index	UNIQUE (user*id) WHERE (status = 'active'::plan*status)
training*plans*user*id*idx	index	(user_id)

4.4.13 plan_sessions

A scheduled session within a plan.

Column	Type	Null	Default
id	uuid	no	
plan_id	uuid	no	
title	text	no	
kind	session_kind	no	
scheduled_for	timestamptz	no	
exercises	smallint	no	
minutes	smallint	no	

Constraint / index	Kind	Definition
plan*session*sexercises_nonnegative	b'c'	CHECK ((exercises >= 0))
plan*session*sminutes_range	b'c'	CHECK (((minutes >= 1) AND (minutes <= 600)))
plan*session*stitle_length	b'c'	CHECK (((char*length(title) >= 1) AND (char*length(title) <= 80)))
plan*session*s*plan**id*fkey	b'f'	FOREIGN KEY (plan*id) REFERENCES training*plans(id) ON DELETE CASCADE
plan*session*spkey	b'p'	PRIMARY KEY (id)
plan*session*s*plan**scheduled*idx	index	(plan*id, scheduled*for)

4.4.14 foods

Seeded foods (30) and users' own foods, macros per 100 g.

Column	Type	Null	Default
id	uuid	no	
name	text	no	
brand	text	yes	
source	food_source	no	
owner*user*id	uuid	yes	
serving_g	numeric(6,1)	no	
kcal	numeric(6,1)	no	
protein_g	numeric(6,1)	no	
carbs_g	numeric(6,1)	no	
fat_g	numeric(6,1)	no	
fibre_g	numeric(6,1)	no	

Constraint / index	Kind	Definition
foods*macros*nonnegative	b'c'	CHECK (((kcal >= (0)::numeric) AND (protein*g >= (0)::numeric) AND (carbs*g >= (0)::numeric) AND (fat*g >= (0)::numeric) AND (fibre*g >= (0)::numeric)))

Constraint / index	Kind	Definition
foods*name*length	b'c'	CHECK (((char*length(name) >= 1) AND (char*length(name) <= 80)))
foods*owner*matches_source	b'c'	CHECK (((source = 'user'::food*source) = (owner*user_id IS NOT NULL)))
foods*serving*positive	b'c'	CHECK ((serving_g > (0)::numeric))
foods*owner*user*id*fkey	b'f'	FOREIGN KEY (owner*user*id) REFERENCES users(id) ON DELETE CASCADE
foods_pkey	b'p'	PRIMARY KEY (id)
foods*owner*user*id*idx	index	(owner*user*id)
foods*seeded*name_key	index	UNIQUE (lower(name)) WHERE (source = 'seeded'::food_source)

4.4.15 meal_entries

A meal eaten at a time.

Column	Type	Null	Default
id	uuid	no	
user_id	uuid	no	
slot	meal_slot	no	
name	text	no	
eaten_at	timestamptz	no	
created_at	timestamptz	no	now()

Constraint / index	Kind	Definition
meal*entries*name_length	b'c'	CHECK (((char*length(name) >= 1) AND (char*length(name) <= 60)))
meal*entries*user*id*fkey	b'f'	FOREIGN KEY (user_id) REFERENCES users(id) ON DELETE CASCADE
meal*entries*pkey	b'p'	PRIMARY KEY (id)
meal*entries*user*eaten*idx	index	(user*id, eaten*at)

4.4.16 meal_items

An item within a meal; macros copied at logging time.

Column	Type	Null	Default
id	uuid	no	
meal_id	uuid	no	
food_id	uuid	yes	
position	smallint	no	
label	text	no	
quantity_g	numeric(7,1)	no	
kcal	numeric(7,1)	no	
protein_g	numeric(6,1)	no	
carbs_g	numeric(6,1)	no	

Column	Type	Null	Default
fat_g	numeric(6,1)	no	
fibre_g	numeric(6,1)	no	

Constraint / index	Kind	Definition
meal*items*label_length	b'c'	CHECK (((char*length(label) >= 1) AND (char*length(label) <= 80)))
meal*items*macros_nonnegative	b'c'	CHECK (((kcal >= (0)::numeric) AND (protein*g >= (0)::numeric) AND (carbs*g >= (0)::numeric) AND (fat*g >= (0)::numeric) AND (fibre*g >= (0)::numeric)))
meal*items*quantity_positive	b'c'	CHECK ((quantity_g > (0)::numeric))
meal*items*food*id*fkey	b'f'	FOREIGN KEY (food_id) REFERENCES foods(id) ON DELETE SET NULL
meal*items*meal*id*fkey	b'f'	FOREIGN KEY (meal*id) REFERENCES meal*entries(id) ON DELETE CASCADE
meal*items*pkey	b'p'	PRIMARY KEY (id)
meal*items*food*id*idx	index	(food_id)
meal*items*meal*id*idx	index	(meal_id)

4.4.17 hydration_logs

Water drunk.

Column	Type	Null	Default
id	uuid	no	
user_id	uuid	no	
logged_at	timestamptz	no	
ml	smallint	no	

Constraint / index	Kind	Definition
hydration*logs*ml_range	b'c'	CHECK (((ml >= 1) AND (ml <= 3000)))
hydration*logs*user*id*fkey	b'f'	FOREIGN KEY (user_id) REFERENCES users(id) ON DELETE CASCADE
hydration*logs*pkey	b'p'	PRIMARY KEY (id)
hydration*logs*user*logged*idx	index	(user*id, logged*at)

4.4.18 nutrition_targets

Calorie, macro and water targets; 1:1 with users.

Column	Type	Null	Default
user_id	uuid	no	
kcal	smallint	no	2200
protein_g	smallint	no	130
carbs_g	smallint	no	250
fat_g	smallint	no	70
fibre_g	smallint	no	30
water_ml	smallint	no	2500

Column	Type	Null	Default
glass_ml	smallint	no	250
weekly*target*kg	numeric(3,2)	no	0
updated_at	timestamptz	no	now()

Constraint / index	Kind	Definition
nutrition*targets*glass_range	b'c'	CHECK (((glass*ml >= 100) AND (glass*ml <= 1000)))
nutrition*targets*kcal_range	b'c'	CHECK (((kcal >= 800) AND (kcal <= 6000)))
nutrition*targets*macro_ranges	b'c'	CHECK ((((protei*g >= 0) AND (protei*g <= 500)) AND ((carbs*g >= 0) AND (carbs*g <= 1000)) AND ((fat*g >= 0) AND (fat*g <= 400)) AND ((fibre*g >= 0) AND (fibre*g <= 150))))
nutrition*targets*water_range	b'c'	CHECK (((water*ml >= 500) AND (water*ml <= 8000)))
nutrition*targets*weekly*target*range	b'c'	CHECK (((weekly*target*kg >= ('-1'::integer)::numeric) AND (weekly*target*kg <= (1)::numeric)))
nutrition*targets*user*id*fkey	b'f'	FOREIGN KEY (user_id) REFERENCES users(id) ON DELETE CASCADE
nutrition*targets*pkey	b'p'	PRIMARY KEY (user_id)

4.5 Enumerated types

Table 25. *PostgreSQL enums (live)*

Enum	Values
access_outcome	authorised, limited, blocked
aggregation	sum, avg, max, last
audit_action	read, share*granted, share*revoked, share_bound
device_kind	phone, wearable, home_monitor, continuous
food_source	seeded, user
grantee_kind	doctor, family, other
meal_slot	breakfast, lunch, dinner, snack
muscle_group	chest, back, shoulders, arms, core, legs, glutes, full_body
plan_status	active, archived
reading_source	healthkit, health_connect, manual
refusal_reason	no*grant, scope*not*granted, revoked, not*yet*valid, expired, email*unconfirmed
revoke_reason	owner, expired*cleanup, grantee*deleted
session_kind	cardio, strength, mobility
sex	female, male, other, undisclosed
share_scope	vitals, activity, sleep, nutrition, records
theme	light, dark, system
units	metric, imperial
workout_type	walk, run, cycle, strength, swim, hiit, yoga, other

4.6 Migrations

Table 26. *Migration history*

Revision	Creates	Notes
0001 baseline	Schema lockdown; enums share*scope, audit*action, access*outcome, refusal*reason; audit_log with triggers	Revokes PUBLIC, anon, authenticated; RLS on
0002 users_profiles	users, profiles; enums sex, units, theme	CHECKs for ranges and formats
0003 health_core	metrics (44 rows), health*readings, ingest*batches, devices; enums reading*source, aggregation, device*kind	Catalogue rows written literally in the migration
0004 consents	consents; enums grantee*kind, revoke*reason	No hospital or app kinds
0005 training	exercises (30 rows), training*plans, plan*sessions, workouts, exercise_sets; 4 enums	Partial unique indexes for one active plan and synced ids
0006 nutrition	foods (30 rows), meal*entries, meal*items, hydration*logs, nutrition*targets; 2 enums	Seeded food ids are uuid5 of the name

Every migration has a working `downgrade()`. CI runs upgrade → downgrade to base → upgrade on every push, and `tests/test_schema_drift.py` compares the SQLAlchemy models with the migrated database using Alembic's autogenerate diff; any difference fails the build.

4.7 Capacity estimate

Table 27. *Capacity*

Item	Estimate
Readings per active user per day	about 310 (heart rate at 5-minute averages = 288, plus about 20 others)
Size per reading including primary-key index	about 120 bytes (no surrogate id, no per-row unit)
Growth per user	about 37 KB a day, 13.5 MB a year
Free database (500 MB) after seeds and indexes	about 30 user-years
Capstone need (15 accounts × 6 months)	about 100 MB
Local demo account (dense 90-day seed incl. a 5-minute glucose sensor)	about 179,000 readings and 127 workouts

5 Class design

Three views of the code as built. The domain model shows the SQLAlchemy classes that map to tables; the service view shows how requests reach them; the consent view shows the classes that guard every shared read. Names match the source files under `backend/app`.

5.1 Domain model: accounts, sharing and audit

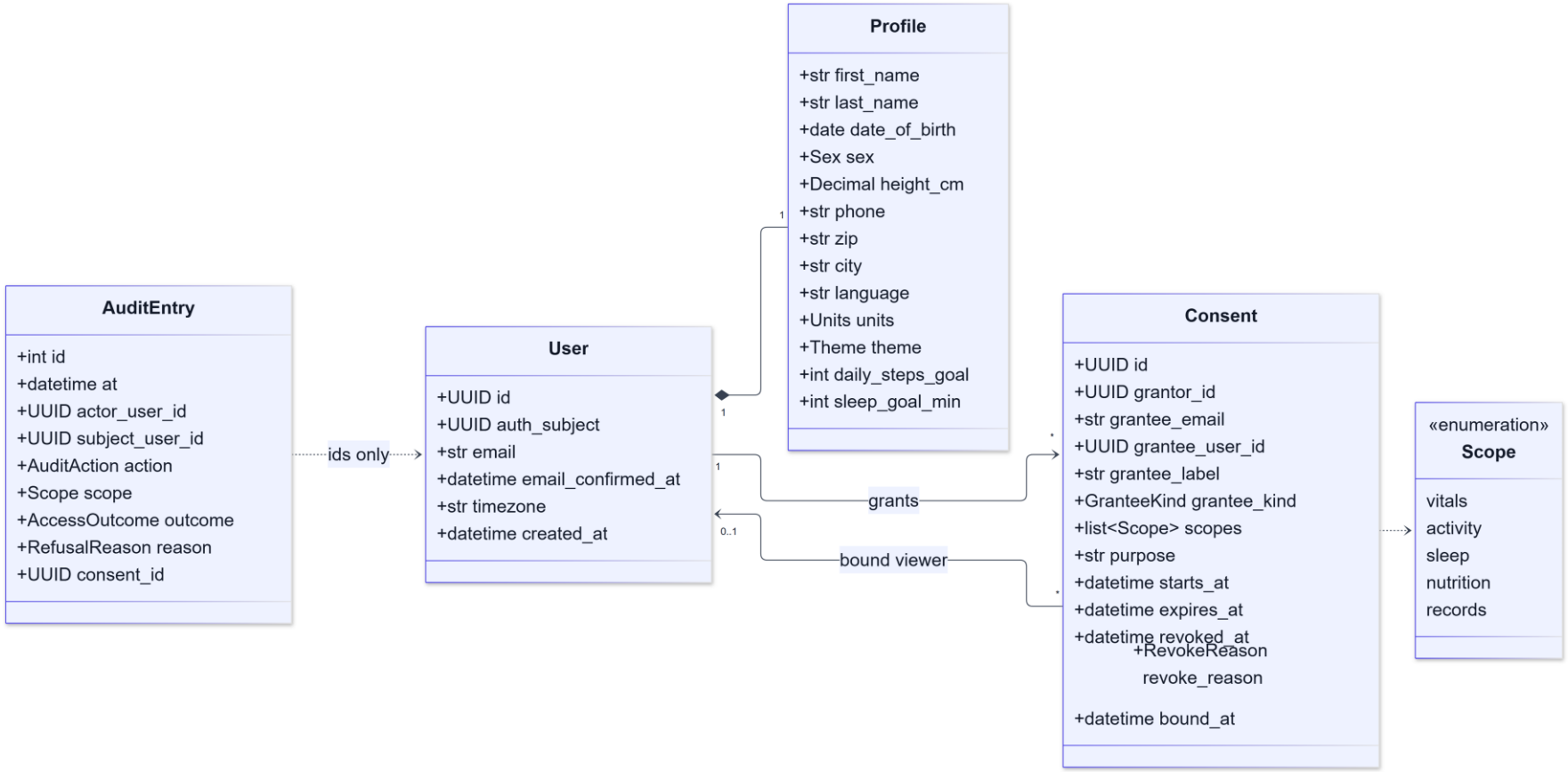


Figure 13. *Class diagram: User, Profile, Consent, AuditEntry*

`User` composes exactly one `Profile`. A `Consent` points to its grantor and, once bound, to its viewer. `AuditEntry` refers to users and consents by id only, with no object references, matching the table's lack of foreign keys.

5.2 Domain model: health, training and nutrition

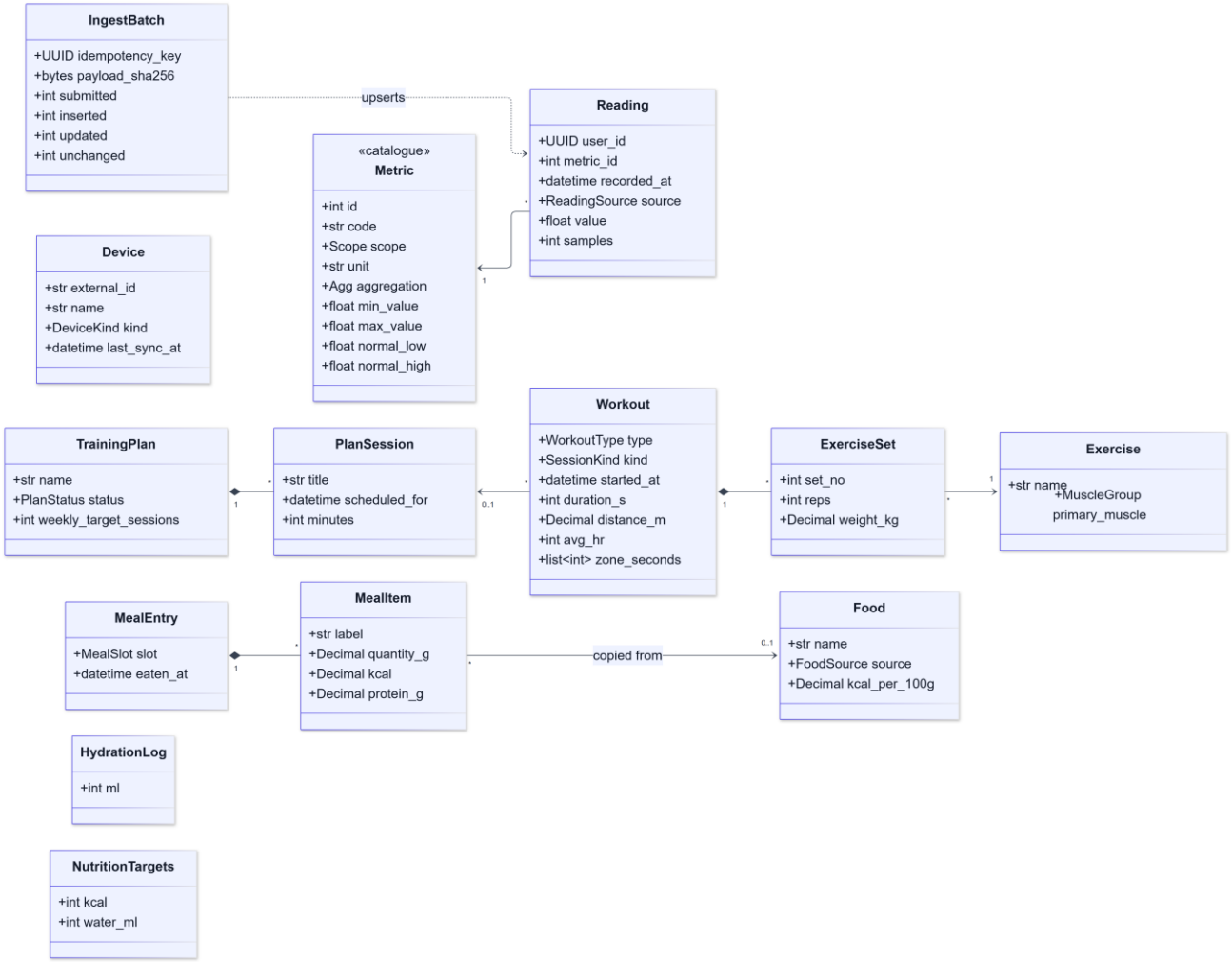


Figure 14. *Class diagram: health, training and nutrition entities*

`Metric` is a frozen dataclass in `catalogue.py`, mirrored by the `metrics` table. A `Workout` composes its `ExerciseSet`s; a `MealEntry` composes its `MealItem`s, which keep copied macros and only optionally reference a `Food`. Every class except the catalogues belongs to one user.

5.3 Service layer

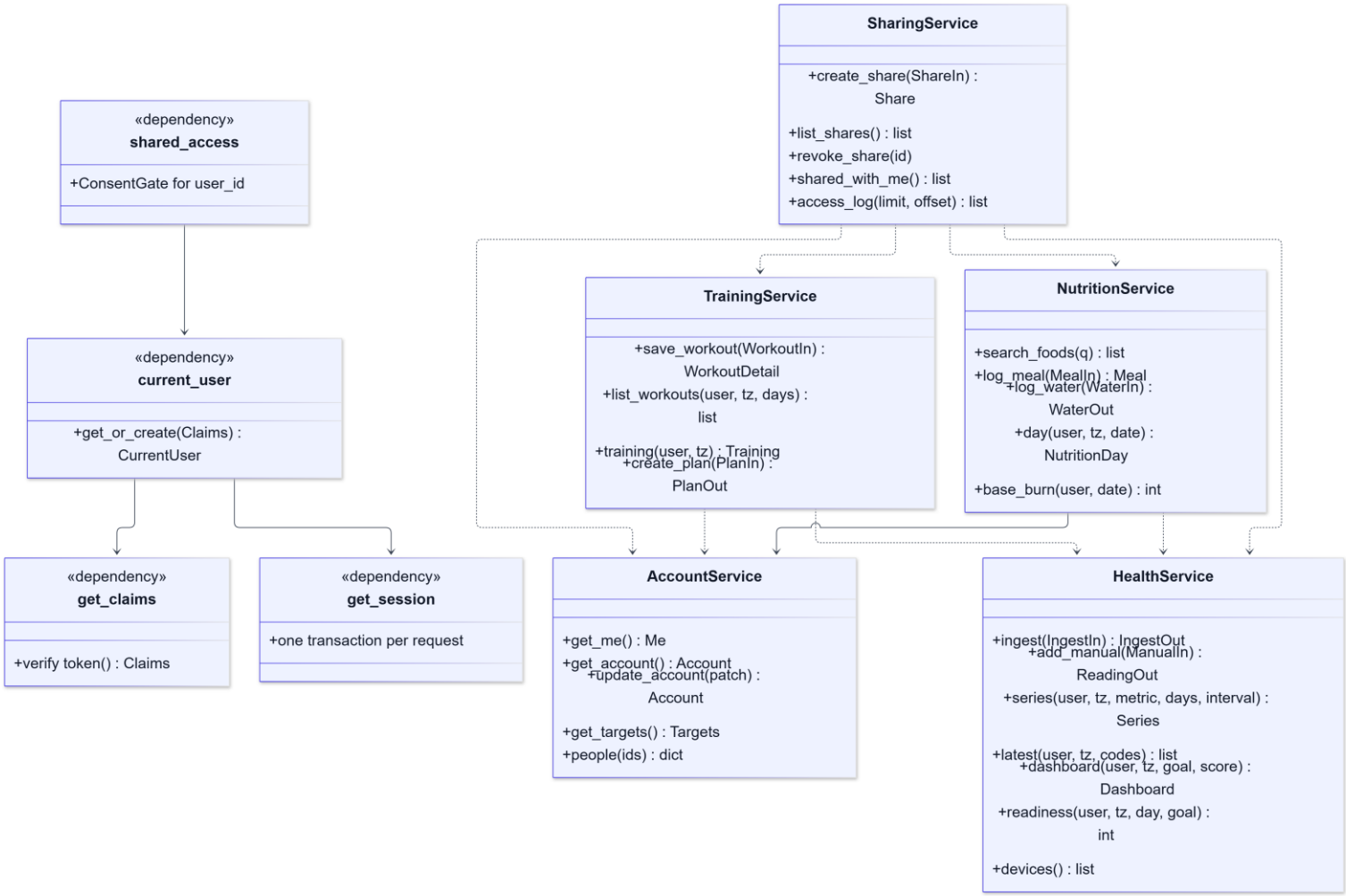


Figure 15. *Class diagram: dependencies and services*

Modules call each other only through `service.py`. The dotted arrows are every cross-module call in the code: sharing reads the other modules to serve shared views; training and nutrition read health totals (active energy, weight) and account data (birth date, body profile).

5.4 Consent engine

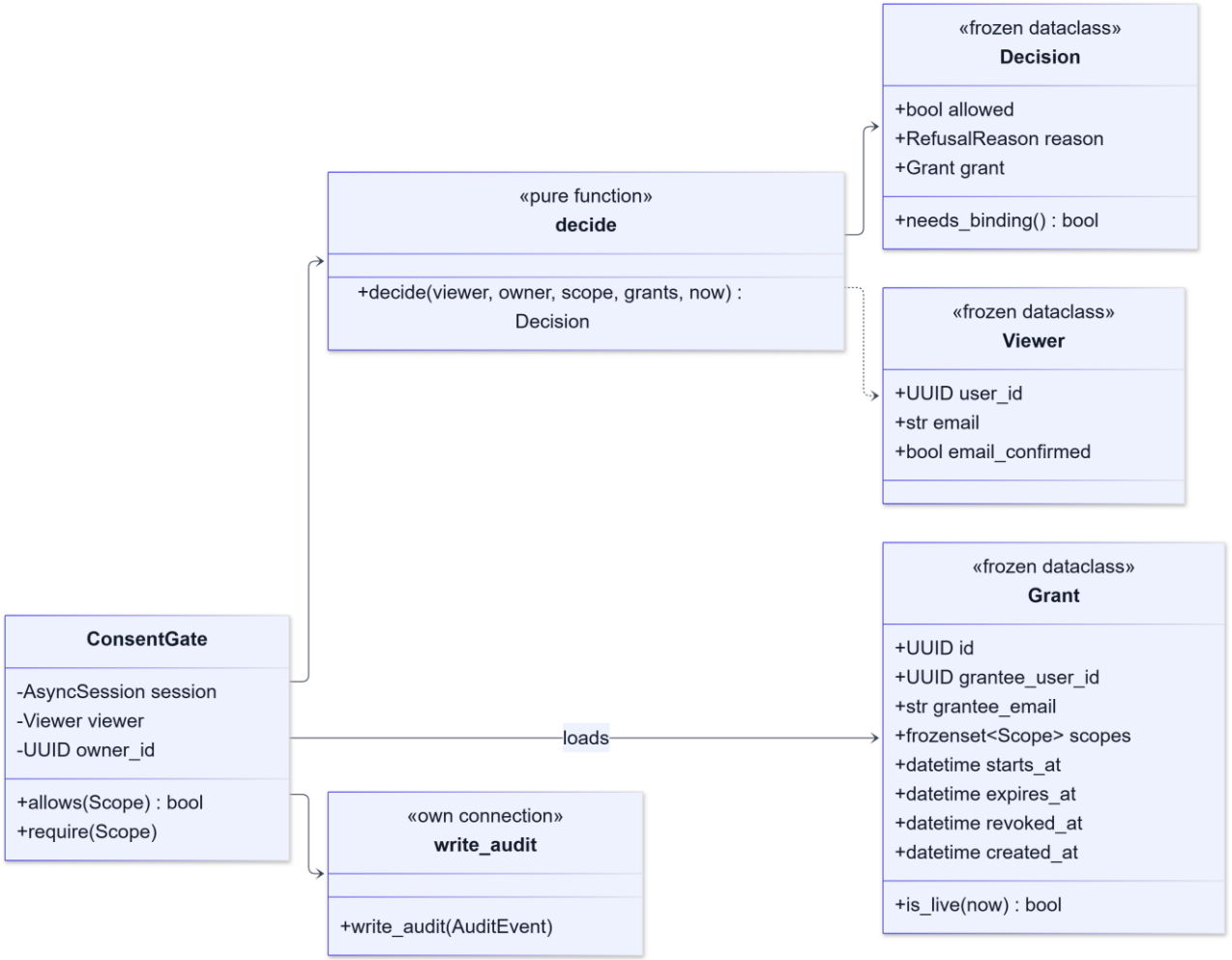


Figure 16. *Class diagram: the consent engine*

`decide()` takes plain values and returns a `Decision`; it has no database, network or framework, so each rule is tested directly. `ConsentGate` wraps it with I/O: it loads the owner's grants for this viewer once per request, calls `decide()`, binds the grant on first allowed use, writes the audit entry through `write_audit()`, and raises 404 or 403 when refused.

6 Processes in detail

Each process below is described in the order the code executes it. Sequence diagrams show the messages between parts; numbered steps explain each one.

6.1 Request lifecycle

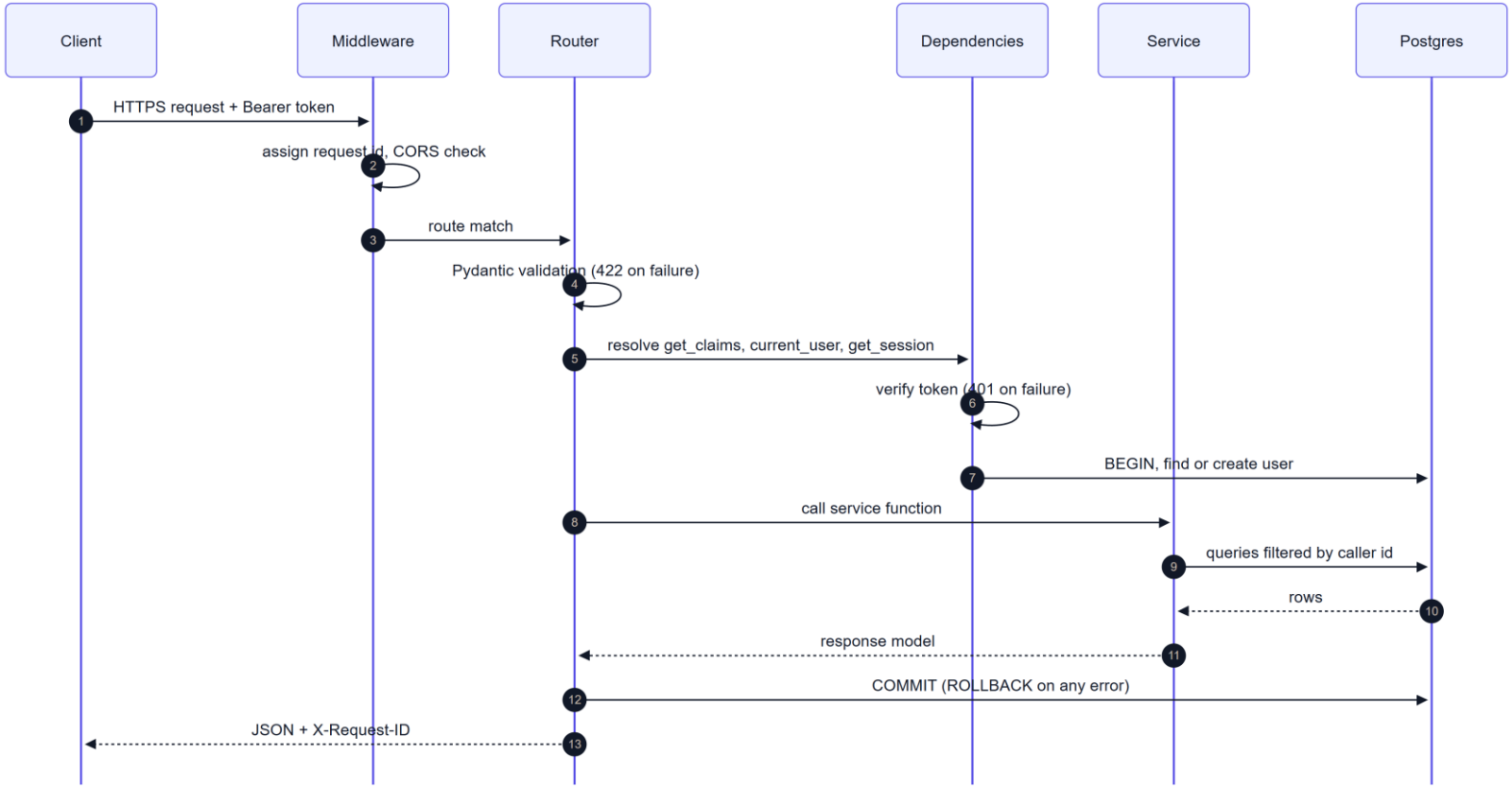


Figure 17. *Sequence: life of any API request*

1. **Request id.** Middleware assigns `req_` + 16 hex characters and returns it in the `X-Request-ID` header and in any error body.
2. **CORS.** Browsers may call only from the configured web-portal origin (`CORS_ORIGIN`).
3. **Routing and validation.** FastAPI matches the route; Pydantic validates path, query and body. Input models forbid unknown fields, so `id`, `email`, `user_id` or `member_since` can never be set by a client. A failure returns 422 `invalid_request` listing field locations only, never the submitted values (which may be health data).
4. **Authentication.** `get_claims` reads `Authorization: Bearer <token>` and verifies it (section 6.2). Missing or invalid → 401 `unauthorized`.
5. **Account.** `current_user` finds or creates the account for the token's subject.
6. **Transaction.** `get_session` opens one database transaction for the whole request. FastAPI caches dependencies per request, so the account lookup and the handler share the same session.
7. **Handler.** The router calls one service function. Every `/me` query filters by the caller's id taken from the token.
8. **Commit or roll back.** If the handler returns, the transaction commits; if anything raises, it rolls back, so nothing is half-written.
9. **Response.** The result is serialised through a response model matching `types.ts`. Errors become `{"error": {"code", "message", "detail", "request_id"}}`; unexpected exceptions become 500 `internal_error` without internal details.

6.2 Sign-in, token verification and account creation

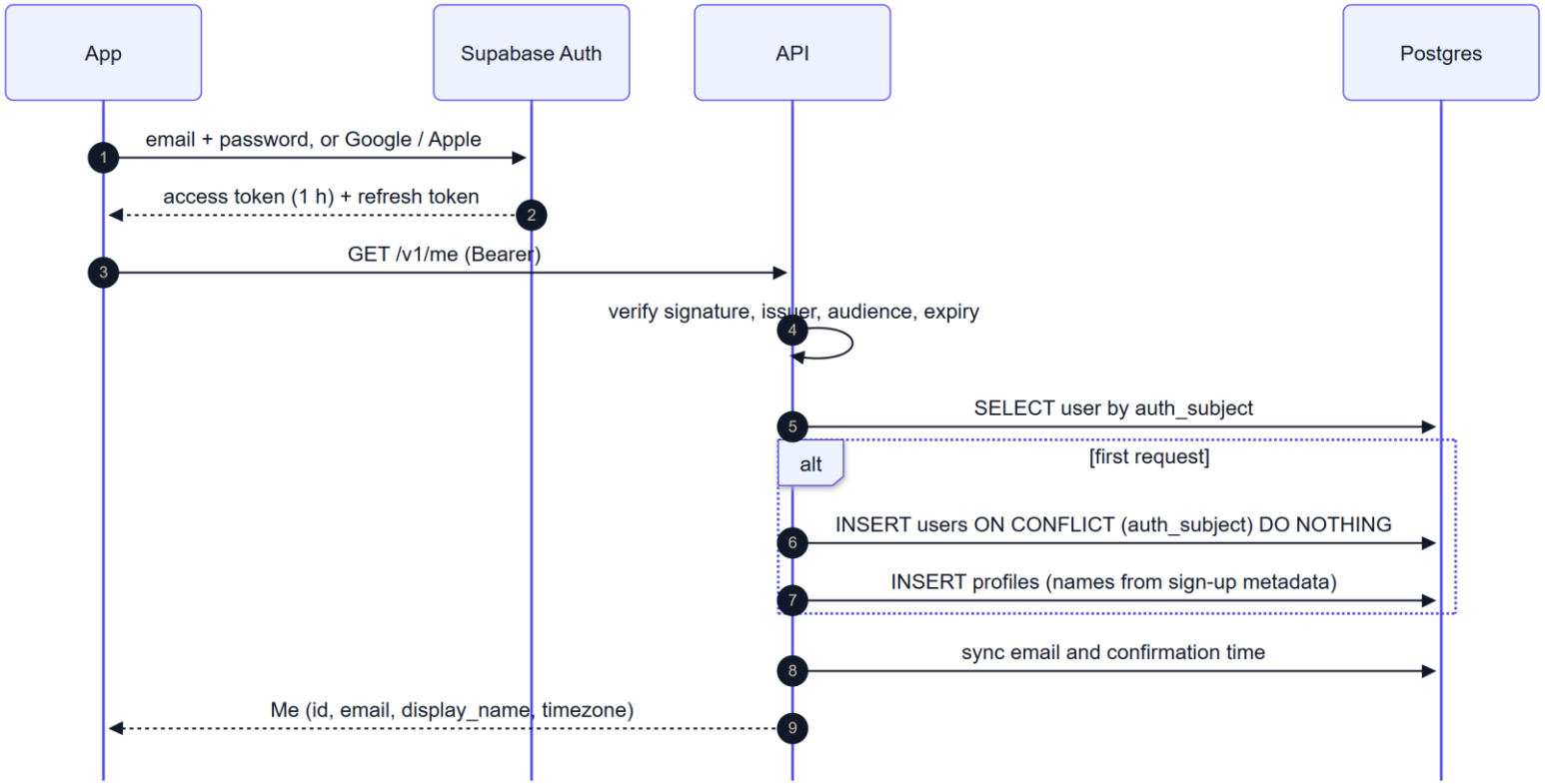


Figure 18. *Sequence: sign-in and first request*

Token verification

1. Split the Authorization header into scheme and token; anything but `Bearer <token>` → 401.
2. **Production:** fetch Supabase's JSON Web Key Set from `<SUPABASE_URL>/auth/v1/.well-known/jwks.json` (cached 10 minutes), pick the key named in the token header, and verify with ES256 or RS256 only. HS256 is never accepted in production.
3. **Local and test:** verify HS256 with `DEV_JWT_SECRET` and issuer `vitallink-dev`.
4. Check issuer, audience (`authenticated`) and required claims `exp`, `iat`, `sub`. Expired → 401 "Your session has expired".
5. Parse claims: subject UUID, lowercased email (missing → 403 `email_required`), `email_verified` from user metadata, assurance level (`aal1`/`aal2`), issue time, and first/last name from sign-up metadata.

Account creation (insert-if-absent)

1. Look up the user by `auth_subject`. If found, skip to step 5.
2. Inside a savepoint, `INSERT INTO users … ON CONFLICT (auth_subject) DO NOTHING RETURNING id`. Several first requests fired at once all succeed; exactly one inserts (tested with six parallel calls).
3. If the insert hits the unique email constraint (another identity owns the email), return 409 `email_in_use`.
4. If this request inserted the user, insert the profile with names from sign-up metadata.
5. If Supabase reports a different email or a newly confirmed email, update the stored email and `email_confirmed_at`, because shares match on confirmed email.

> Locally, `python -m app.devtoken <email>` prints a token shaped like Supabase's. The subject is a uuid5 of the email, so the same email always maps to the same account.

6.3 Phone sync (ingest)

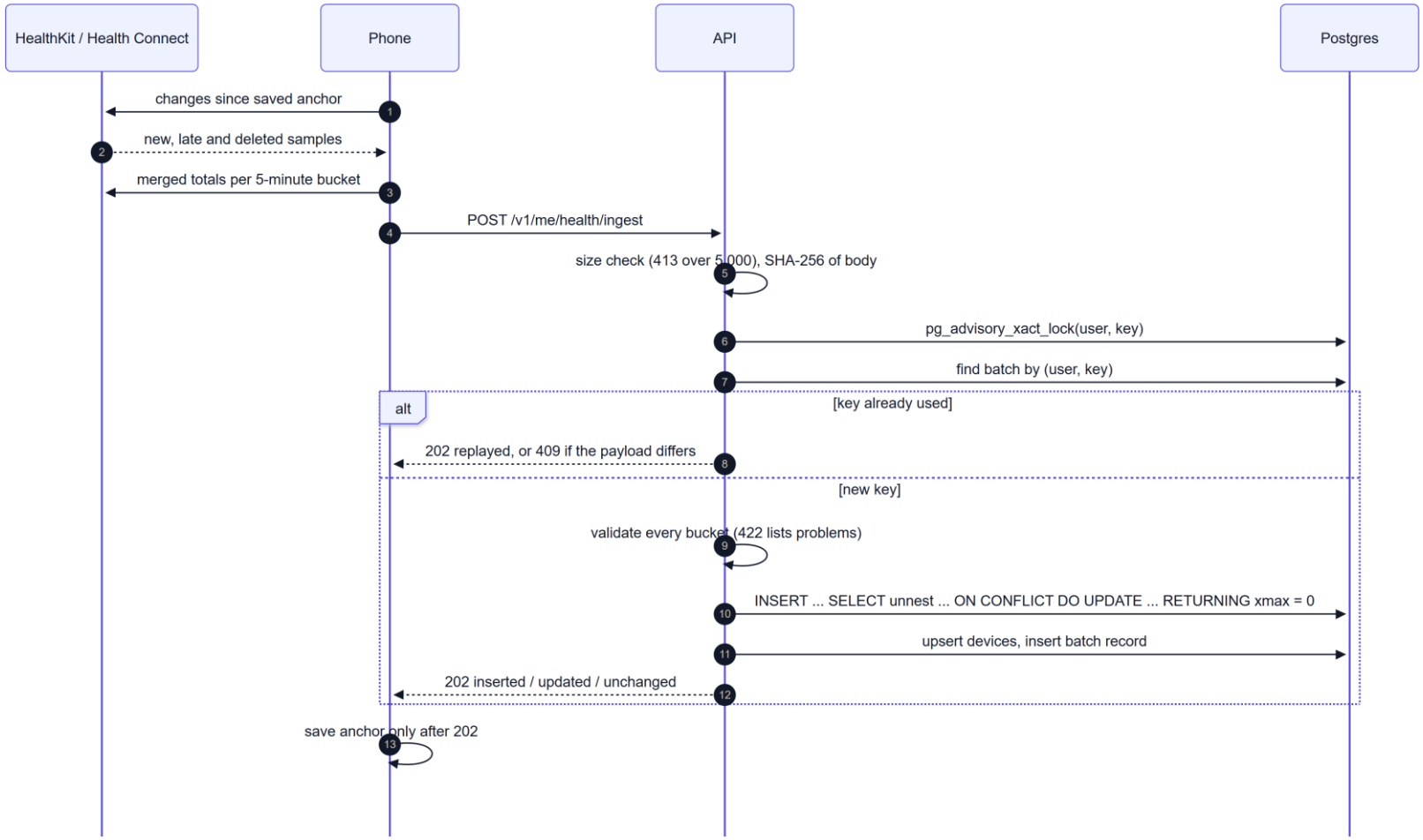


Figure 19. *Sequence: phone sync upload*

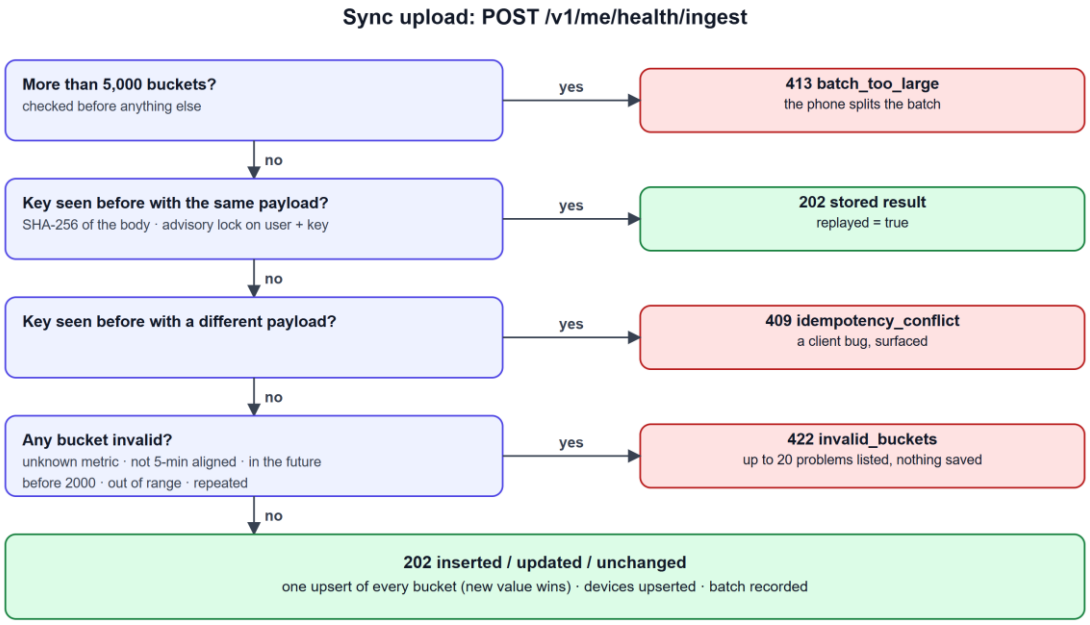


Figure 20. *Decision ladder: how an upload is accepted or refused*

1. **Size check.** More than 5,000 buckets → 413 `batch_too_large`; the phone splits the batch.
2. **Fingerprint.** SHA-256 of the canonical JSON of the request body.
3. **Serialise retries.** `pg_advisory_xact_lock(hashtextextended('ingest:<user>:<key>', 0))` makes a retry that races the original wait for it instead of applying twice. The lock is released at commit.
4. **Replay check.** Look up `ingest_batches` by (user, key). Same hash → return the stored counts with `replayed: true`. Different hash → 409 `idempotency_conflict`, surfacing a client bug instead of silently ignoring data.
5. **Validate every bucket.** Known metric; start on a 5-minute boundary (`epoch % 300 = 0`); not more than 5 minutes in the future; not before 2000; value inside the metric's plausible range (e.g. heart rate 25–250 bpm, steps 0–5,000 per bucket); no repeated (metric, start) pair. Devices may list only known metrics. Any failure → 422 `invalid_buckets` with up to 20 problems by index, and nothing is saved.
6. **Upsert.** One statement: `INSERT INTO health_readings … SELECT FROM unnest(metric_ids, times, values, samples) ON CONFLICT (user_id, metric_id, recorded_at, source) DO UPDATE SET value, samples, updated_at WHERE (value, samples) IS DISTINCT FROM (new) RETURNING (xmax = 0) AS inserted`.
7. **Count.** Rows returned with `xmax = 0` were inserted; other returned rows were updated; rows not returned were unchanged (the WHERE clause skipped them).
8. **Devices.** Upsert each device by (user, platform, external id), setting name, kind, metric ids and `last_sync_at = now()`.
9. **Batch record.** Insert the batch with its counts; a CHECK guarantees inserted + updated + unchanged = submitted.
10. **Respond 202.** The phone advances its anchor only after this response.

Why upsert rather than insert-once: watch data often reaches the phone hours late and users can delete bad samples, so an already-uploaded bucket can change. The anchored query reports those changes, the phone

re-sends the affected buckets, and the new value replaces the old one. Keeping the first value would freeze wrong totals forever.

6.4 Manual readings

1. Resolve the metric (unknown → 404) and require `manual_allowed` (e.g. weight, mood, cuff blood pressure, waist; not steps) → else 422 `not_manual`.
2. Reject future times (beyond 5 minutes) and values outside the plausible range.
3. Floor the time to its 5-minute bucket and upsert with source `manual` (a second entry in the same bucket replaces the first).
4. Return the reading with its assessment (Normal / High / Low where the metric has a normal range).

6.5 Time series

1. Resolve the metric in the catalogue (unknown → 404).
2. Compute the window in the user's IANA time zone: from local midnight `days - 1` days ago to now. For a Dallas user "today" runs midnight to midnight Central time.
3. In one SQL query, `generate_series` produces every bucket in the window (hours, days, ISO weeks starting Monday, or months) as local timestamps.
4. Readings in the window are grouped by `date_trunc(interval, recorded_at AT TIME ZONE tz)` and aggregated by the catalogue's rule: `sum` for totals, `avg` for vitals, last value for VO2 max.
5. The two are left-joined. Empty buckets: totals show 0; averages are skipped, so a chart never shows a fake zero heart rate.
6. Values are rounded by unit (whole numbers for bpm, %, mg/dL, mmHg, counts, kcal, minutes; one decimal otherwise) and bucket times are returned with the user's offset.
7. `blood_pressure` returns two series (systolic and diastolic), built the same way from the paired metrics.

6.6 Dashboard, assessments and readiness

1. For heart rate, oxygen saturation, weight and glucose: the latest reading, its time, the 7-day average and a range status (low, normal, high, or no-data) from the catalogue's normal range.
2. For blood pressure: the latest systolic/diastolic pair recorded at the same instant from the same source, rated by American Heart Association category.
3. Readiness is computed for today and for 7 days ago from up to three parts; at least two are required, otherwise the score is null.
4. The 7-day change is today's score minus last week's.

Table 28. *Blood pressure categories (checked top to bottom)*

Category	Systolic (mmHg)	Diastolic (mmHg)	Range status
High (stage 2)	≥ 140	or ≥ 90	high
High (stage 1)	130–139	or 80–89	high
Elevated	120–129	and < 80	borderline
Low	< 90	or < 60	low
Normal	otherwise		normal

Readiness parts:

$$\text{sleep} = \min(100,\ 100 \times \text{minutes asleep from 18:00 the evening before to 14:00} \div \text{sleep goal})$$

$$\text{resting HR} = \text{clamp}_0{}^{100}(\ 100 - 10 \times \max(0,\ \text{RHR today} - \text{RHR 30-day average})\ )$$

$$\text{HRV} = \text{clamp}_0{}^{100}(\ 100 \times \text{HRV today} \div \text{HRV 30-day average}\ )$$

$$\text{readiness} = \text{round}(\ \text{mean of the available parts}\ ),\ \text{if at least two are available}$$

Shared dashboards (viewed by someone else) leave the readiness score out, because it uses sleep data the viewer may not have been given.

6.7 Devices

1. List the user's devices, most recently synced first.
2. Label the data each provides from the catalogue (heart rate from a home monitor is shown as "Pulse"); wearables also get a 3-item short list for mobile.
3. Choose an icon by kind and metrics: wearable, continuous sensor (droplet), scale, blood-pressure monitor, or generic.
4. Mark "overdue" when the last sync is older than 24 hours (phones, wearables, continuous sensors) or 3 days (home monitors).
5. Forgetting a device deletes only the device row; its readings stay, because they belong to the user.

6.8 Sharing and the consent decision

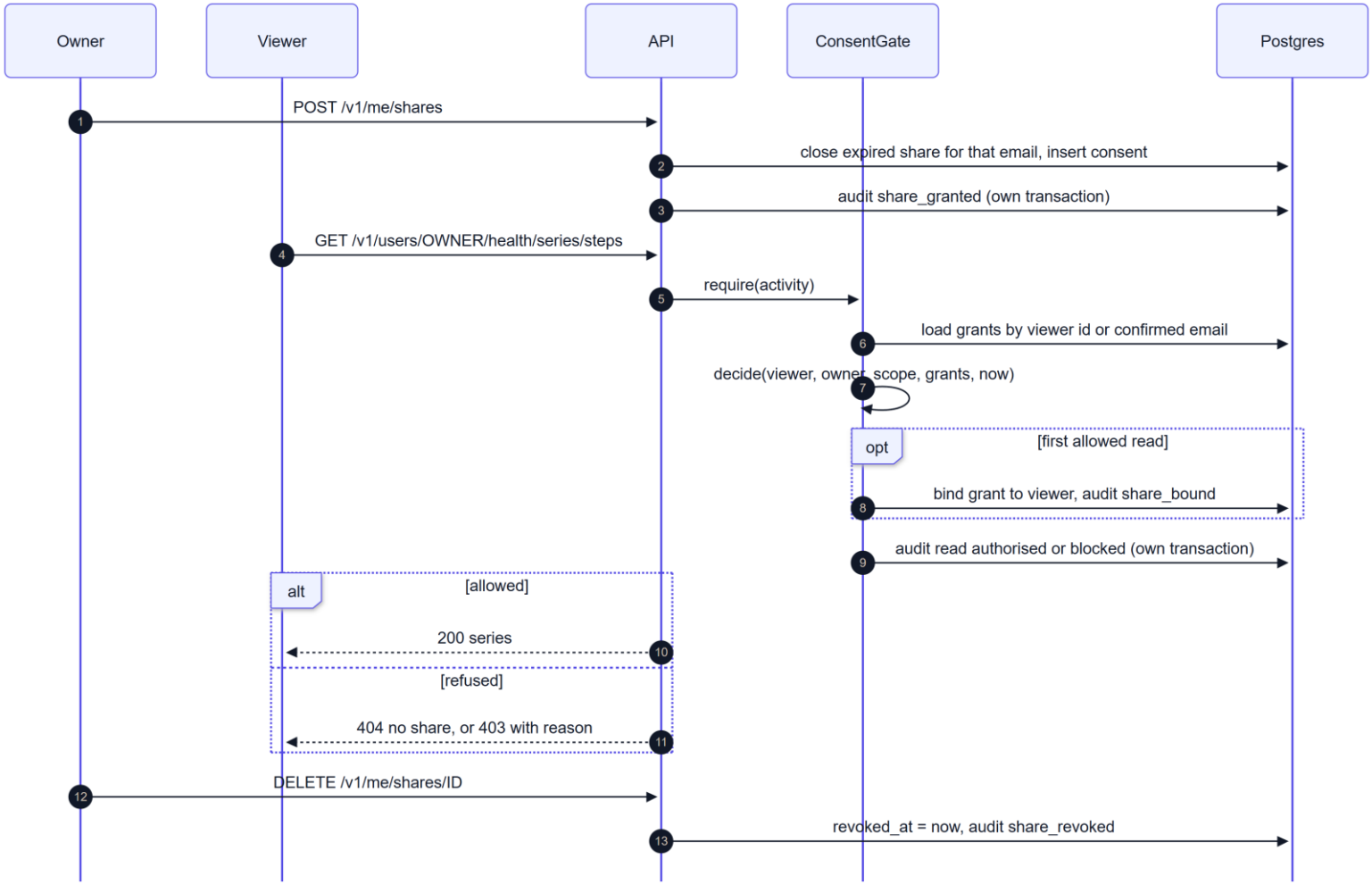


Figure 21. *Sequence: creating a share, a shared read, and revocation*

Creating a share

1. Lowercase the email. Sharing with your own email → 422 `cannot_share_with_self`.
2. Validate the window: start defaults to now; an expiry must be after the start and in the future.
3. Close any expired-but-unrevoked share to the same email (`revoke_reason = expired_cleanup`).
4. If a live (unrevoked) share to that email exists → 409 `share_exists` (the partial unique index enforces the same rule in the database).
5. Insert the consent with sorted, de-duplicated scopes and write a `share_granted` audit entry.

Deciding a shared read

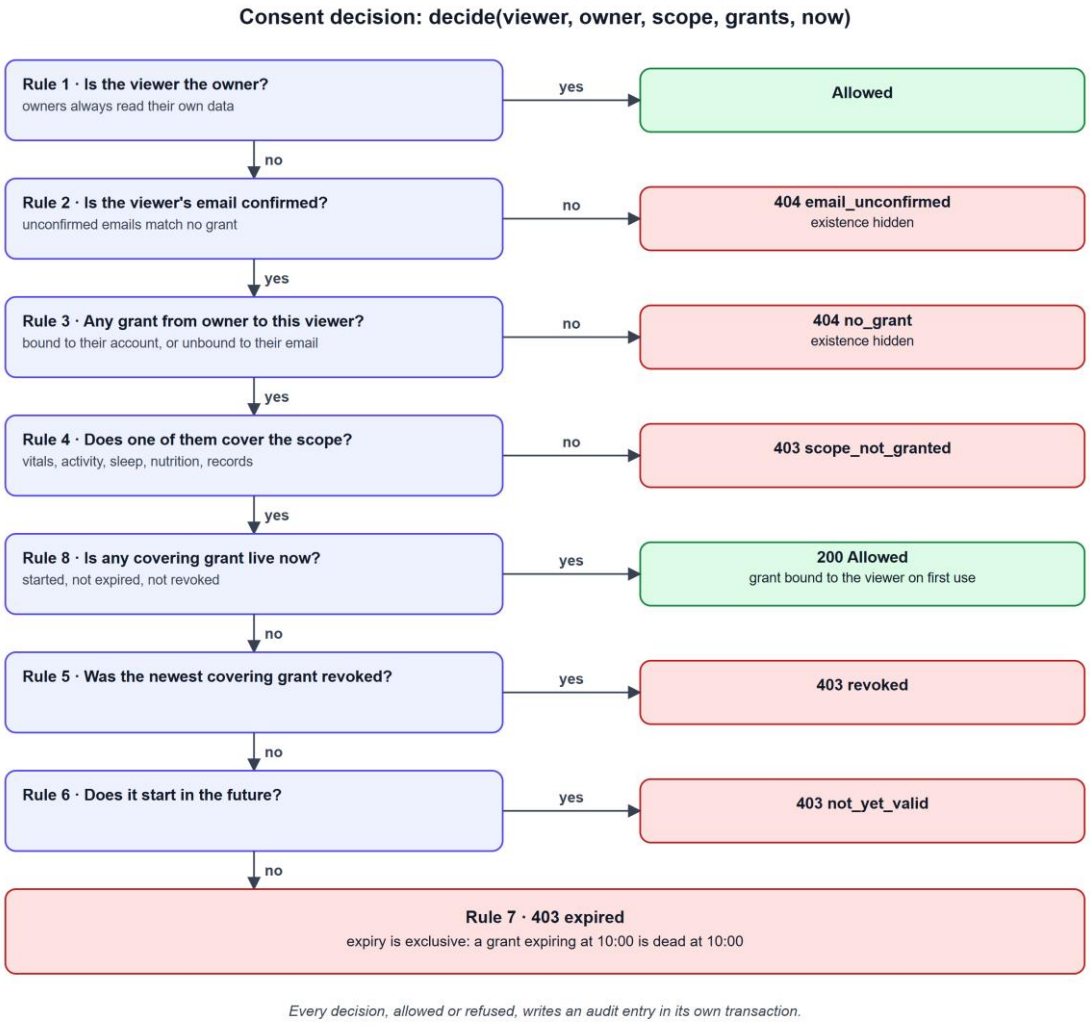


Figure 22. *Decision ladder: the eight consent rules*

1. Every `/v1/users/{id}/…` route depends on `shared_access`, which builds a `ConsentGate` for (viewer, owner, request id).
2. The handler calls `gate.require(scope)`; the scope comes from the route (e.g. activity for workouts) or from the requested metric's catalogue entry.
3. The gate loads, once per request, the owner's grants that match this viewer: bound to the viewer's account id, or unbound and addressed to the viewer's email.

4. `decide()` applies rules 1–8 (figure above) and returns allowed or a refusal reason; for refusals it names the newest relevant grant so the owner's log shows who asked.
5. On the first allowed read through an unbound grant, the gate sets `grantee_user_id` and `bound_at` and writes a `share_bound` audit entry. From then on the share follows the account, not the email.
6. The gate writes a `read` audit entry (authorised or blocked, with scope, reason and consent id) on its own connection.
7. Refused with `no_grant` or `email_unconfirmed` → 404 `not_found`, indistinguishable from a user that doesn't exist. Other refusals → 403 with the reason as the error code.

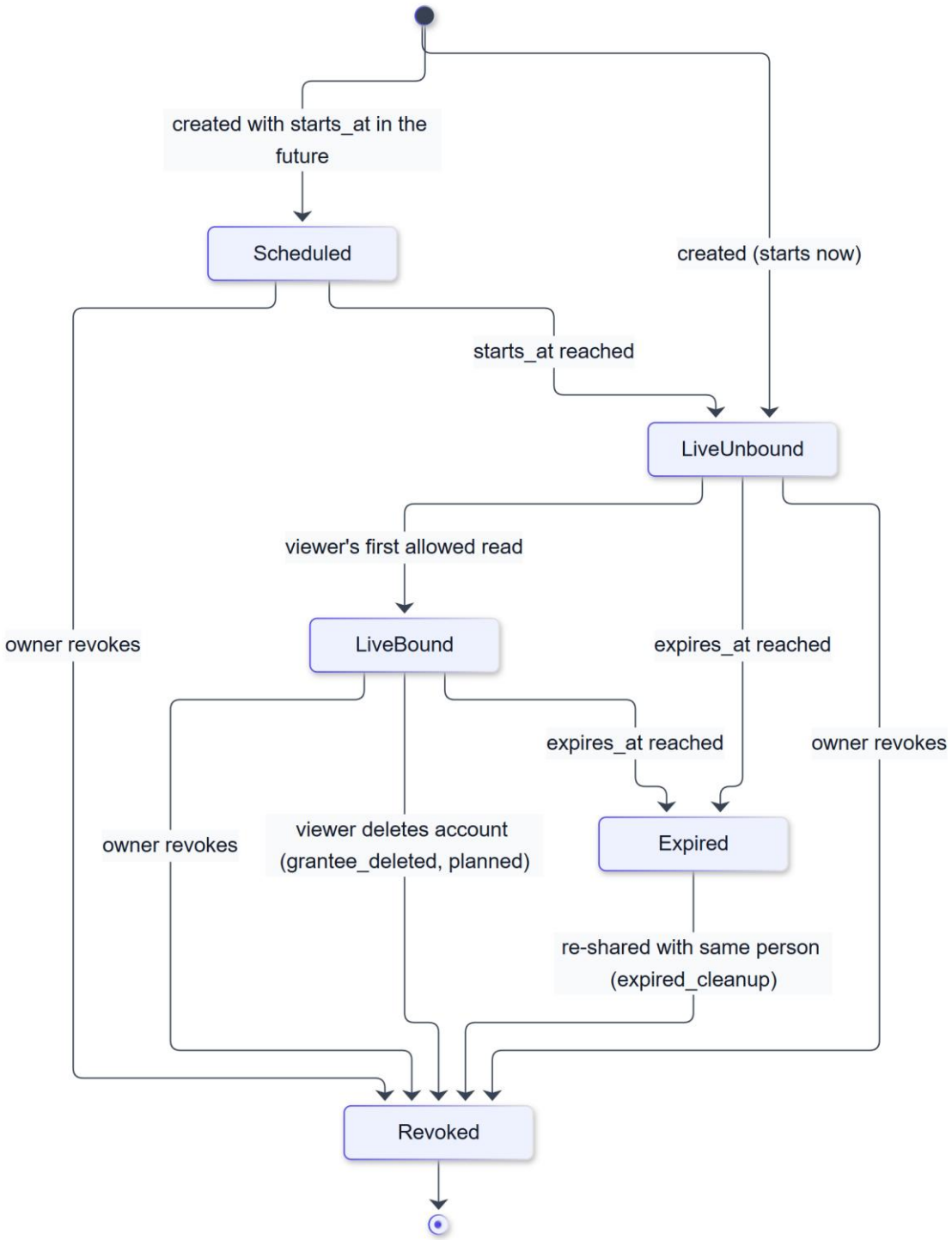


Figure 23. *State diagram: the life of a share*

Why the audit write has its own transaction

A refusal raises an error, and an error rolls back the request's transaction. If the audit entry lived in that transaction, the refusals, which are the accesses most worth recording, would vanish. `write_audit()` therefore opens its own connection and commits before the error is raised. If the audit write itself fails, it is logged and the access decision stands.

Revocation and the access log

Revoking sets `revoked_at` and `revoke_reason = owner` (only on the owner's own, unrevoked shares; otherwise 404) and writes `share_revoked`. It takes effect on the viewer's next request because nothing is cached. The access log returns reads of the owner's data by others, newest first; each entry is joined to its share for the person's label, kind and stated purpose, and a refusal reason is translated into words (e.g. "Share revoked", "Not part of the share").

6.9 Training

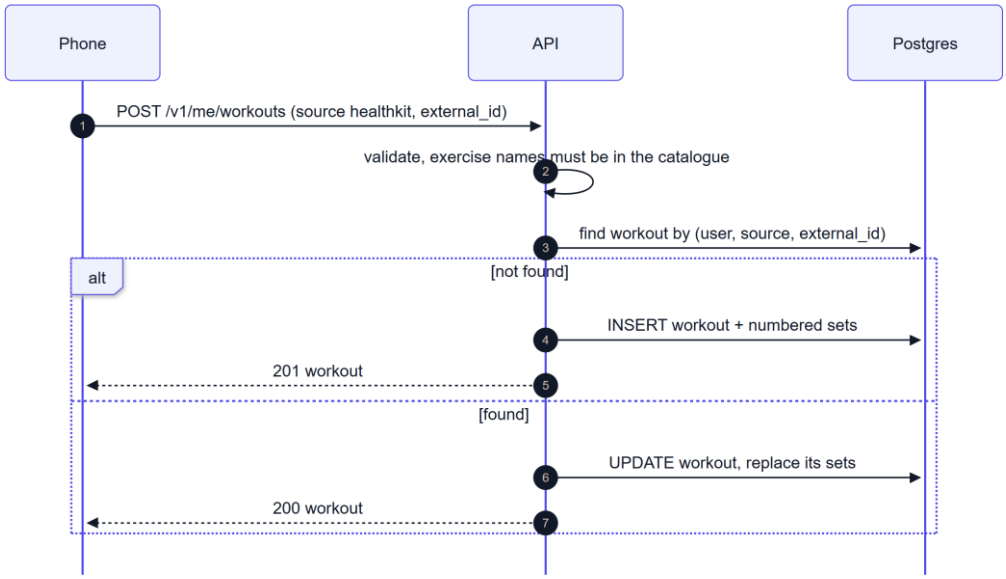


Figure 24. *Sequence: logging a synced workout*

1. **Validate.** Exercise names must be in the 30-exercise catalogue (case-insensitive); a set needs reps or a duration; synced workouts must carry an external id and manual ones must not.
2. **Plan link.** A `plan_session_id` must belong to one of the caller's plans, else 404.
3. **Upsert.** A synced workout with a known (source, external id) is updated in place and its sets replaced (200); otherwise a new workout is created (201). Kind defaults from type: strength → strength, yoga → mobility, others → cardio.
4. **Number sets** per exercise in the order sent (1, 2, 3…).

The training view

1. Load the last 365 days of workouts with their sets in one query (no N+1).
2. **Volume** = Σ reps × weight over weighted sets. **Pace** (runs and walks) = duration ÷ distance in km.
3. **Zone** = the zone with most recorded time; otherwise average heart rate as a share of maximum (220 − age, or 190 without a birth date): < 60% zone 1, < 70% zone 2, < 80% zone 3, < 90% zone 4, else 5; with no heart rate, mobility = 1, others = 3.
4. **Personal records** (figure below).
5. **Week:** sessions, minutes, volume and per-day load by kind for the current Monday-to-Sunday week; active energy summed from health readings; source = the most recently synced wearable's name.
6. **Streaks:** current = consecutive days with a workout ending today or yesterday (today isn't over yet); longest = the longest run in the year.

7. **Muscle balance:** sets per primary muscle group over 14 days; any group under 40% of the most-trained group is named as trailing.
8. **Next session:** the earliest planned session after now in the active plan; the weekly target comes from that plan (default 5).

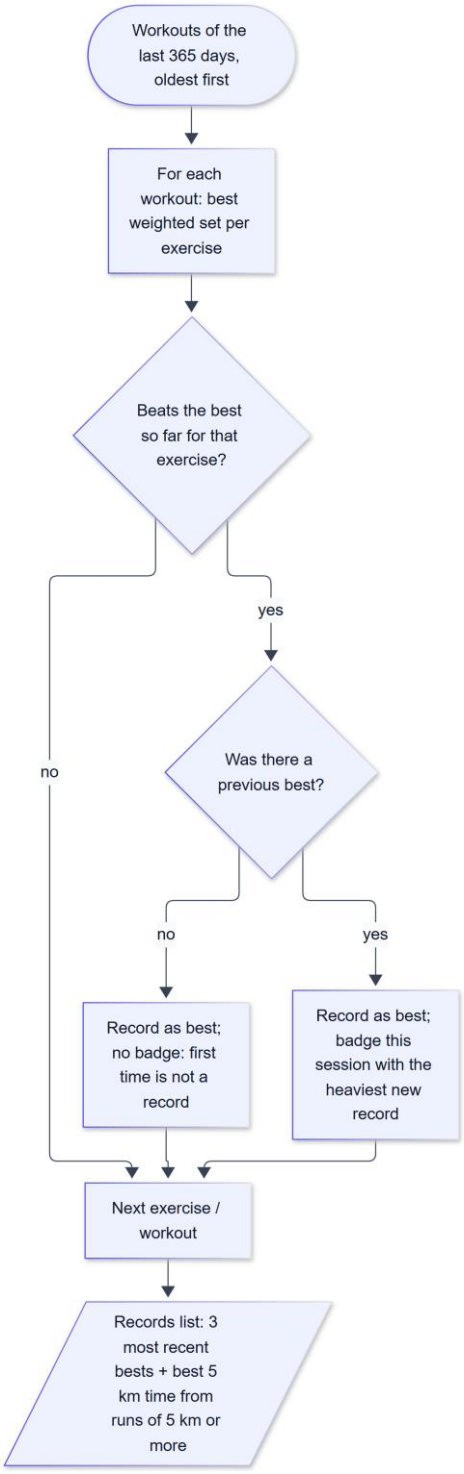


Figure 25. *Activity: personal record detection*

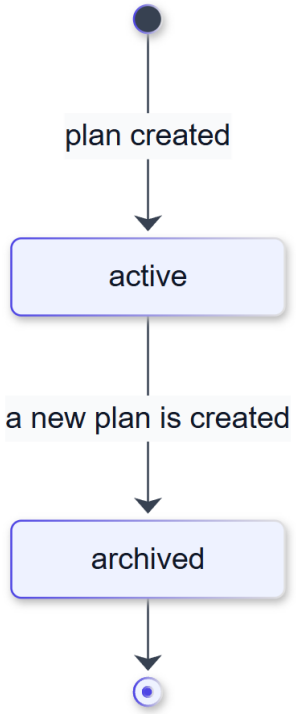


Figure 26. *State diagram: training plan status*

6.10 Nutrition

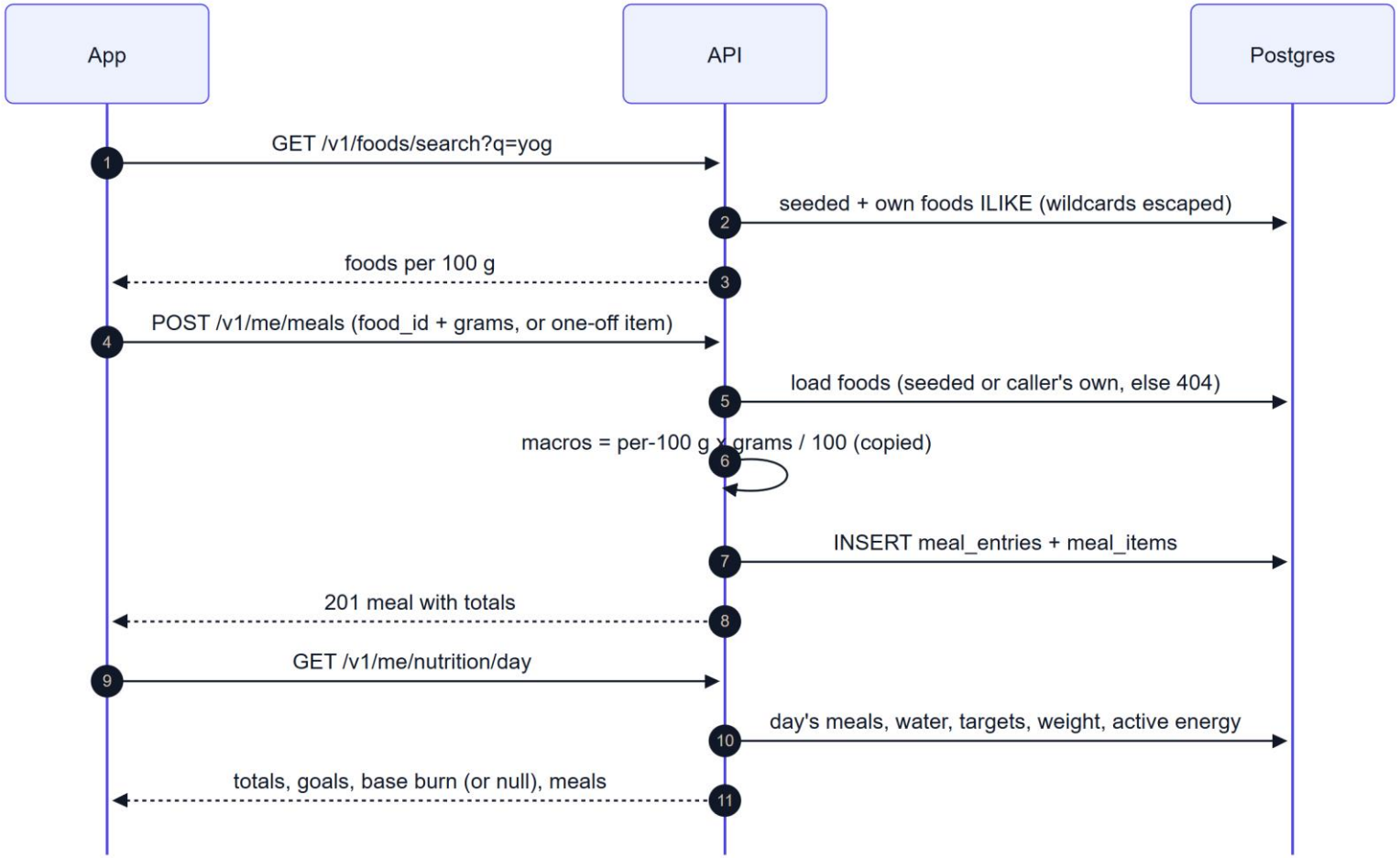


Figure 27. *Sequence: food search, meal logging and the day view*

1. **Search** seeded foods and the caller's own by name with `ILIKE`; `%` and `_` in the query are escaped so they match literally; shortest names first, at most 25.
2. **Log a meal.** Each item is either a food plus grams or a one-off item with its own macros (sending both, or neither, → 422). Foods must be seeded or the caller's own (else 404).
3. **Copy macros:** for a food item, each macro = value per 100 g × grams ÷ 100, rounded to 0.1. The copy is stored on the item, so editing or deleting a food never rewrites past meals.
4. **Targets** are created with defaults on first read (insert-if-absent): 2,200 kcal, 130 g protein, 250 g carbs, 70 g fat, 30 g fibre, 2,500 ml water, 250 ml glass.
5. **Day view:** sum the local day's meal items and water; add active energy from health readings; estimate resting burn (below); return the meals in time order.

$$\text{BMR} = 10 \times \text{weight(kg)} + 6.25 \times \text{height(cm)} - 5 \times \text{age} + \text{s};\quad \text{s} = +5\ \text{(male)}, -161\ \text{(female)}, -78\ \text{(other / undisclosed)}$$

The base burn is null until height, the latest weight reading, birth date and sex are all known; the screen then asks for the missing details instead of showing a guess.

6.11 Planned processes

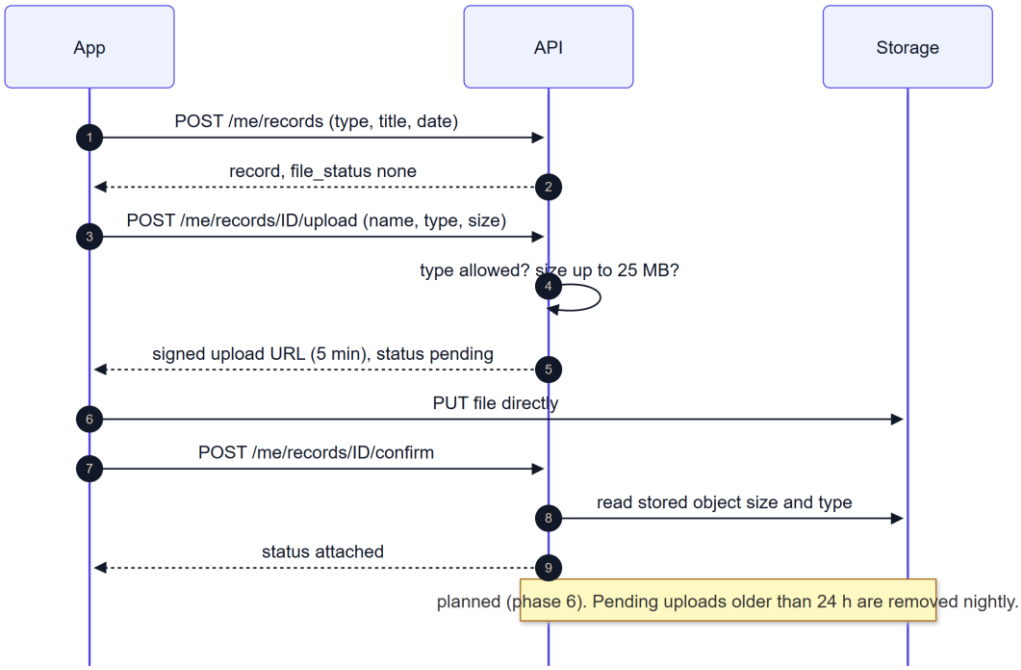


Figure 28. *Sequence (planned): document upload with signed URLs*

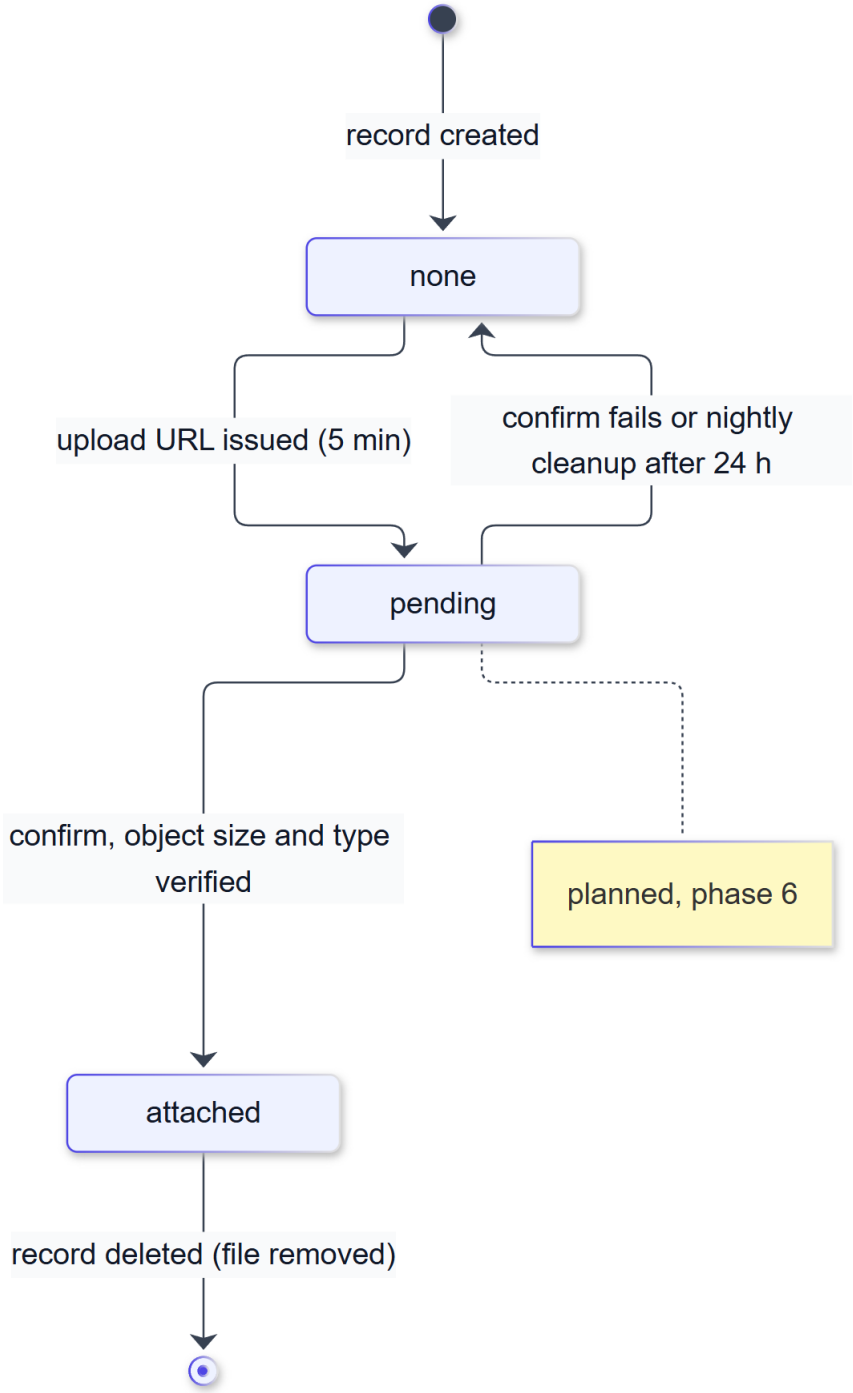


Figure 29. *State diagram (planned): a record's file status*

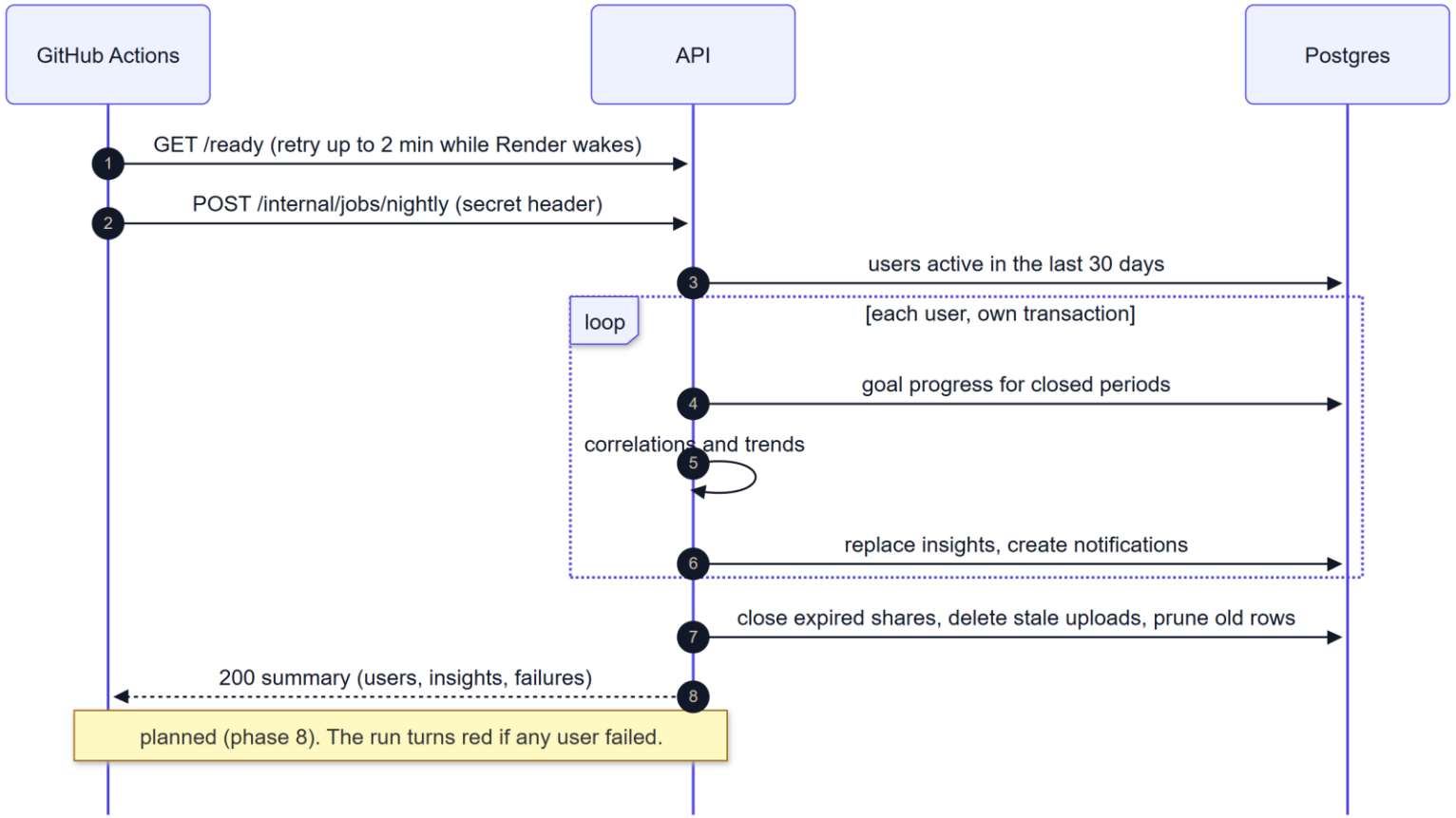


Figure 30. *Sequence (planned): the nightly job*

Table 29. *Other planned processes*

Process	Steps
Account deletion	1. Revoke and unbind shares received (`grantee_deleted`). 2. Delete stored files. 3. Delete the user (cascades to all owned rows). 4. Delete the Supabase identity. Audit entries remain as ids and enums.
Export	Collect every row the user owns from each module's export hook into one JSON download.
Dose reminders	Local notifications scheduled on the phone (no push service, so no health text passes through Apple, Google or Expo); the server records taken or skipped doses and flags missed ones nightly.
Insights	Per user nightly: daily series for candidate metric pairs; correlations with at least 14 days of data and a multiple-comparison correction; wording as associations, never causes.

7 API specification

The API exposes **44 operations** today (42 under `/v1`, plus `/health` and `/ready`). FastAPI generates the OpenAPI description at `/openapi.json` and interactive documentation at `/docs`; the tables below are generated from that description.

7.1 Conventions

Table 30. *API conventions*

Topic	Rule
Route families	`/v1/me/…` = the caller's own data. `/v1/users/{user_id}/…` = someone else's data, read-only, every route behind the consent gate. `/v1/foods`, `/v1/exercises` = shared catalogues.
Authentication	`Authorization: Bearer <token>` on everything except /health and /ready. No cookies, so there is no CSRF surface.
Content	JSON in and out; ISO 8601 times with offsets; SI units
Idempotency	Ingest takes an `idempotency_key`; synced workouts are keyed by platform id; dose logging (planned) by its natural key
Pagination	`limit` (1–200, default 50) and `offset` on the access log
Versioning	Additive changes stay in /v1; breaking changes go to /v2

7.2 Status and error codes

Table 31. *Status and error codes*

HTTP	Code	When
200	—	Read or update succeeded; synced workout updated
201	—	Created (manual reading, share, workout, meal, food, water, plan)
202	—	Ingest accepted (also for a replay)
204	—	Deleted or revoked
401	unauthorized	Missing, malformed, expired or invalid token
403	email_required	Token has no email
403	scope*not*granted · revoked · not*yet*valid · expired	Shared read refused by the consent engine
404	not_found	Unknown resource, someone else's resource, or an owner who hasn't shared with you
404	unknown*metric* · *unknown*food	Not in the catalogue / not visible to you
409	email*in*use · idempotency*conflict* · *share*exists	Conflicts
413	batch*too*large	Ingest over 5,000 buckets
422	invalid*request* · *invalid*buckets · unknown*exercise* · *not*manual · cannot*share*with_self	Validation failures
500	internal_error	Unexpected failure; details only in server logs
503	db_unavailable	Database unreachable (/ready)

```
{"error": {"code": "scope_not_granted",
           "message": "This share doesn't include vitals data.",
```

```
          "detail": {"scope": "vitals"},
          "request_id": "req_912a02fbebae43bc"}}
```

7.3 Endpoint reference

Table 32. *Operations endpoints*

Method	Path	Input	Success	Description
GET	/health		200	The process is up.
GET	/ready		200	The database is reachable. The uptime monitor pings this, which also keeps Supabase awake.

Table 33. *Account endpoints*

Method	Path	Input	Success	Description
GET	/v1/me		200	The signed-in account. The first call creates it.
GET	/v1/me/account		200	Get Account
PATCH	/v1/me/account	body: AccountPatch	200	Update Account
GET	/v1/me/targets		200	Get Targets
PATCH	/v1/me/targets	body: TargetsPatch	200	Update Targets

Table 34. *Health endpoints*

Method	Path	Input	Success	Description
POST	/v1/me/health/ingest	body: IngestIn	202	Batch of 5-minute aggregates from HealthKit or Health Connect (≤ 5,000), upserted.
POST	/v1/me/health/manual	body: ManualIn	201	Add Manual
GET	/v1/me/health/dashboard		200	Dashboard
GET	/v1/me/health/series/blood_pressure	days (query) interval (query)	200	Blood Pressure
GET	/v1/me/health/series/{metric}	metric (path) days (query) interval (query)	200	Buckets in the user's time zone. Totals fill empty buckets with 0; averages skip them.
GET	/v1/me/health/latest	metrics (query)	200	Latest reading per metric (comma-separated codes; `blood_pressure` pairs the two values).
GET	/v1/me/devices		200	Devices
DELETE	/v1/me/devices/{device_id}	device_id (path)	204	Forget Device

Table 35. *Sharing (owner side) endpoints*

Method	Path	Input	Success	Description
POST	/v1/me/shares	body: ShareIn	201	Share chosen scopes with a person, read-only, from `starts_at` until `expires_at` or revocation.
GET	/v1/me/shares		200	List Shares
DELETE	/v1/me/shares/{share_id}	share_id (path)	204	Revoke Share
GET	/v1/me/shared-with-me		200	Shared With Me

Method	Path	Input	Success	Description
GET	/v1/me/access-log	limit (query) offset (query)	200	Access Log

Table 36. *Shared views (viewer side, consent-checked) endpoints*

Method	Path	Input	Success	Description
GET	/v1/users/{user_id}/health/dashboard		200	Vitals only; the readiness score is left out because it also uses sleep data.
GET	/v1/users/{user*id}/health/series/blood*pressure	days (query) interval (query)	200	Shared Blood Pressure
GET	/v1/users/{user_id}/health/series/{metric}	metric (path) days (query) interval (query)	200	Shared Series
GET	/v1/users/{user_id}/health/latest	metrics (query)	200	Shared Latest
GET	/v1/users/{user_id}/workouts	days (query)	200	Shared Workouts
GET	/v1/users/{user_id}/training		200	Shared Training
GET	/v1/users/{user_id}/nutrition/day	date (query)	200	Shared Nutrition Day

Table 37. *Training endpoints*

Method	Path	Input	Success	Description
POST	/v1/me/workouts	body: WorkoutIn	201	Log a workout with its sets. A synced workout with a known platform id is updated (200).
GET	/v1/me/workouts	days (query)	200	List Workouts
GET	/v1/me/workouts/{workout_id}	workout_id (path)	200	Get Workout
DELETE	/v1/me/workouts/{workout_id}	workout_id (path)	204	Delete Workout
GET	/v1/me/training		200	This week's load, recent sessions, muscle balance, personal records and the next planned session.
POST	/v1/me/training/plans	body: PlanIn	201	Create Plan
GET	/v1/me/training/plans		200	List Plans
GET	/v1/exercises		200	Exercises

Table 38. *Nutrition endpoints*

Method	Path	Input	Success	Description
GET	/v1/foods/search	q (query)	200	Search Foods
POST	/v1/me/foods	body: FoodIn	201	Create Food
POST	/v1/me/meals	body: MealIn	201	Macros are copied from the food at logging time.
DELETE	/v1/me/meals/{meal_id}	meal_id (path)	204	Delete Meal
POST	/v1/me/hydration	body: WaterIn	201	Log Water
DELETE	/v1/me/hydration/{log_id}	log_id (path)	204	Delete Water
GET	/v1/me/nutrition/day	date (query)	200	Totals, targets, water, energy balance and meals for one local day (default today).
GET	/v1/me/nutrition/targets		200	Get Targets

Method	Path	Input	Success	Description
PATCH	/v1/me/nutrition/targets	body: NutritionTargetsIn	200	Update Targets

7.4 Request and response bodies

Selected request and response models, generated from the OpenAPI schemas. Required fields are marked.

Table 39. *Schema IngestIn*

Field	Type	Required
idempotency_key	string (uuid)	yes
platform	healthkit | health_connect	yes
devices	list[DeviceIn]	
buckets	list[BucketIn]	yes

Table 40. *Schema BucketIn*

Field	Type	Required
metric	string	yes
start	string (date-time)	yes
value	number	yes
samples	integer	

Table 41. *Schema DeviceIn*

Field	Type	Required
external_id	string	yes
name	string	yes
kind	DeviceKind	yes
metrics	list[string]	

Table 42. *Schema IngestOut*

Field	Type	Required
batch_id	integer	yes
submitted	integer	yes
inserted	integer	yes
updated	integer	yes
unchanged	integer	yes
replayed	boolean	yes

Table 43. *Schema ManualIn*

Field	Type	Required
metric	string	yes
value	number	yes
at	string (date-time) | null	

Table 44. *Schema ReadingOut*

Field	Type	Required
metric	string	yes
value	number	yes
value2	number | null	
unit	string	yes
at	string (date-time) | null	yes
assessment	string	yes

Table 45. *Schema MetricSummary*

Field	Type	Required
metric	string	yes
unit	string	yes
latest	number | null	yes
latest_at	string (date-time) | null	yes
avg_7d	number | null	yes
range	normal | borderline | high | low | no-data	yes

Table 46. *Schema Series*

Field	Type	Required
metric	string	yes
unit	string	yes
interval	hour | day | week | month	yes
points	list[SeriesPoint]	yes

Table 47. *Schema AccountPatch*

Field	Type	Required
first_name	string | null	
last_name	string | null	
date*of*birth	string (date) | null	
sex	Sex | null	
height_cm	number | null	
phone	string | null	
zip	string | null	
city	string | null	
language	en-US | es-US | null	
units	Units | null	
theme	Theme | null	

Field	Type	Required
timezone	string | null	

Table 48. *Schema TargetsPatch*

Field	Type	Required
daily*steps*goal	integer | null	
move*kcal*goal	integer | null	
exercise*min*goal	integer | null	
stand*hours*goal	integer | null	
sleep*goal*min	integer | null	

Table 49. *Schema ShareIn*

Field	Type	Required
email	string (email)	yes
label	string	yes
kind	GranteeKind	yes
scopes	list[Scope]	yes
purpose	string	
starts_at	string (date-time) | null	
expires_at	string (date-time) | null	

Table 50. *Schema Share*

Field	Type	Required
id	string (uuid)	yes
grantee_label	string	yes
grantee_email	string	yes
grantee_kind	GranteeKind	yes
scopes	list[Scope]	yes
purpose	string	yes
starts_at	string (date-time)	yes
expires_at	string (date-time) | null	yes
active	boolean	yes

Table 51. *Schema SharedWithMe*

Field	Type	Required
id	string (uuid)	yes
owner_id	string (uuid)	yes
owner_name	string	yes
scopes	list[Scope]	yes
expires_at	string (date-time) | null	yes

Table 52. *Schema AccessLogEntry*

Field	Type	Required
at	string (date-time)	yes
actor	string	yes
actor_detail	string	yes
accessed	string	yes
purpose	string	yes
status	authorised | limited | blocked	yes

Table 53. *Schema WorkoutIn*

Field	Type	Required
type	WorkoutType	yes
kind	SessionKind | null	
title	string	yes
started_at	string (date-time)	yes
duration_s	integer	yes
distance_m	number | null	
kcal	integer | null	
avg_hr	integer | null	
rpe	integer | null	
zone_seconds	list[integer] | null	
focus	string | null	
source	manual | healthkit | health_connect	
external_id	string | null	
plan*session*id	string (uuid) | null	
sets	list[SetIn]	

Table 54. *Schema SetIn*

Field	Type	Required
exercise	string	yes
reps	integer | null	
weight_kg	number | null	
duration_s	integer | null	

Table 55. *Schema TrainingWeek*

Field	Type	Required
sessions	integer	yes
target_sessions	integer	yes

Field	Type	Required
minutes	integer	yes
volume_kg	number	yes
active_kcal	integer	yes
streak_days	integer	yes
longeststreakdays	integer	yes
source	string	yes
load	list[object]	yes

Table 56. *Schema MealIn*

Field	Type	Required
slot	MealSlot	yes
name	string | null	
eaten_at	string (date-time)	yes
items	list[ItemIn]	yes

Table 57. *Schema ItemIn*

Field	Type	Required
food_id	string (uuid) | null	
quantity_g	number	yes
label	string | null	
kcal	number | null	
protein_g	number | null	
carbs_g	number | null	
fat_g	number | null	
fibre_g	number | null	

Table 58. *Schema NutritionDay*

Field	Type	Required
date	string (date-time)	yes
kcal	integer	yes
kcal_goal	integer	yes
protein_g	integer	yes
carbs_g	integer	yes
fat_g	integer	yes
fibre_g	integer	yes
goals	MacroGoals	yes
water_ml	integer	yes
watergoalml	integer	yes
glass_ml	integer	yes

Field	Type	Required
base*burn*kcal	integer | null	yes
active*burn*kcal	integer	yes
weekly*target*kg	number	yes
meals	list[Meal]	yes

7.5 Contract with the frontend

`packages/api/src/types.ts` holds one TypeScript interface per endpoint and the Pydantic response models match it. Client calls whose endpoint exists use `get()` and are live: me, account, targets, dashboard, series, blood pressure, latest, devices, shares, shared-with-me, access log, workouts, training and nutrition. The rest use `pending()`: mock mode serves the Figma sample data, and live mode makes the screen say "isn't available yet" so a real user never sees invented numbers.

8 Security and privacy

The API is the only door to the data: four database locks stop anyone going around it, token checks decide who is calling, and the consent engine decides what they may read.

8.1 Database lockdown

Supabase publishes a REST API over exposed schemas, and the `anon` key inside the mobile app is public by design. If the tables were reachable that way, anyone could bypass the API, the consent engine and the audit log. Migration 0001 and the tests enforce four locks:

1. All tables live in the private `vitallink` schema, not in the Data API's exposed list.
2. The Data API is switched off for the project (production setting); Supabase is used only for Auth and Storage.
3. Row-level security is enabled on every table with no policies (deny all); a test checks every table.
4. `REVOKE ALL` on the schema from PUBLIC, and from anon and authenticated (including default privileges) where those roles exist.

8.2 Authentication and authorisation

- Tokens are verified on every request (signature, issuer, audience, expiry); production accepts only ES256/RS256 from Supabase's JWKS; HS256 dev tokens work only when ENV is local or test (a test proves production refuses a correctly signed HS256 token).
- Email confirmation is required; an unconfirmed email matches no share.
- Every `/me` query filters by the user id from the verified token, never from the path or body.
- Every `/users/{id}` route depends on `shared_access`; a guardrail test walks all routes to prove it, and another proves those routes are GET-only.
- Unknown and unshared users return the same 404, and UUIDv7 ids are not guessable, so ids can't be used to discover accounts.
- Accepted risk: a signed-out token stays valid until it expires, at most 1 hour.

8.3 Threats and controls

Table 59. *Threat model*

Threat	Control
Reading the database with the public app key	The four locks in 8.1
Claiming a share by signing up with someone else's email	Email confirmation; unconfirmed emails match nothing; grants bind to the account on first use
Inheriting a deleted viewer's shares	FK RESTRICT forces account deletion to revoke and unbind them first
IDOR on /me	Ownership filter from the token on every query; tests try other users' ids
IDOR on /users/{id}	Consent gate on every route, enforced by a test
Mass assignment	Input models forbid unknown fields
Forged token	Signature, issuer, audience, expiry; no HS256 in production
SQL injection	Bound parameters through SQLAlchemy; the only interpolated SQL is a fixed aggregate from a closed set; search wildcards escaped

Threat	Control
Abusive sync	5,000-bucket cap, plausibility ranges, future-time limit, idempotency
Health data in logs	Request bodies never logged; validation errors echo field names, not values
Rewriting history	Audit triggers block UPDATE, DELETE and TRUNCATE; entries hold no free text
Secrets in the repository	.env git-ignored; .env.example holds placeholders; secrets in Render and GitHub
Health text through push providers	No remote push; reminders are local notifications on the phone

8.4 Privacy commitments and legal context

- **Data minimisation:** no location history, contacts or advertising ids; sensor data stored as 5-minute aggregates, never raw samples.
- **Nothing inbound, nothing to third parties:** no other system writes into a user's record; no analytics or advertising services; sharing only sends data out, read-only, to a person the owner names. Share kinds are doctor, family or other; there is no hospital or third-party-app kind.
- **Export and delete** (planned): one call exports everything; one deletes rows, files and the sign-in identity.
- **Legal:** with no provider involved, HIPAA does not apply; the FTC Health Breach Notification Rule does. An unauthorised disclosure, not only a hack, is a breach, and affected users must be told within 60 days. The audit log is what lets the team say exactly whose data was touched.

9 Testing and quality assurance

134 test cases (92 test functions, several run with multiple inputs) pass in about 29 seconds against a real PostgreSQL 16. Tests use a real database, never a mock, because the guarantees that matter most (de-duplication, cascades, CHECK constraints, the append-only trigger) live in the database.

9.1 Test environment

1. `tests/conftest.py` derives a separate `<name>_test` database from `DATABASE_URL`, creates it if missing, and migrates it to head once per session.
2. Each test signs in fresh users with dev tokens (`make_token`), so tests never share data and run in any order.
3. If PostgreSQL isn't running, database tests are skipped locally; in CI (`REQUIRE_DB=1`) they fail instead.

9.2 Test suites

Table 60. *Test suites*

File	Functions	What it proves
test*consent*rules.py	14	Every consent rule with plain values: owner; unconfirmed email; no grant; bound grant follows the account; scope not granted; revoked; not yet valid; expiry exclusive; live grant wins; vitals never opens records
test_sharing.py	16	Grant → read → revoke → refused end to end, all in the access log; email change keeps access, old address gets nothing; refusals audited despite rollback; share validation (no hospital or app); shared training needs activity; guardrails
test_health.py	21	Catalogue equals database; no blood terms; insert/update/unchanged counts; replay and 409; whole batch rejected on one bad bucket; 413; local-midnight buckets; averages skip gaps; weeks start Monday; BP pairs and categories; manual readings; dashboard and readiness; devices; isolation
test_training.py	9	Sets numbered; synced workout updated not duplicated; volume, pace, zone; PRs only after a previous best; 5 km record; streaks; muscle note; plans archive the old; no linking another user's plan session
test_nutrition.py	9	Food catalogue equals database; search returns seeded and own foods only, wildcards literal; macros copied; another user's food refused; day totals by local day; Mifflin–St Jeor only with a full profile; target ranges
test_account.py	10	Account on first call with a UUIDv7; email lowercased; six parallel first calls create one account; 409 for a reused email; defaults; patch and clear; ten rejected patches; isolation; deleting a user cascades
test_security.py	5	Missing token 401; expired, wrong audience, wrong issuer, wrong key, no expiry, garbage → 401; no email 403; no dev tokens in production; production refuses HS256
test_database.py	7	/ready; nothing in public; RLS on every table; audit log refuses UPDATE, DELETE, TRUNCATE; reason only when blocked
test*schema*drift.py	1	SQLAlchemy models and Alembic migrations describe the same schema

9.3 Guardrails

Table 61. *Guardrail tests*

Constraint	Test
Consumers only	No route path contains hospital, clinic, provider or admin; share kinds hospital and app are rejected
No blood features	No metric code contains blood*type, blood*group, donor or blood_bank; a "Blood Donation" exercise is rejected
Consent on every shared read	Every /v1/users route depends on shared_access
Viewers never write	Shared routes allow only GET
History can't be rewritten	Audit UPDATE, DELETE, TRUNCATE raise

9.4 Continuous integration

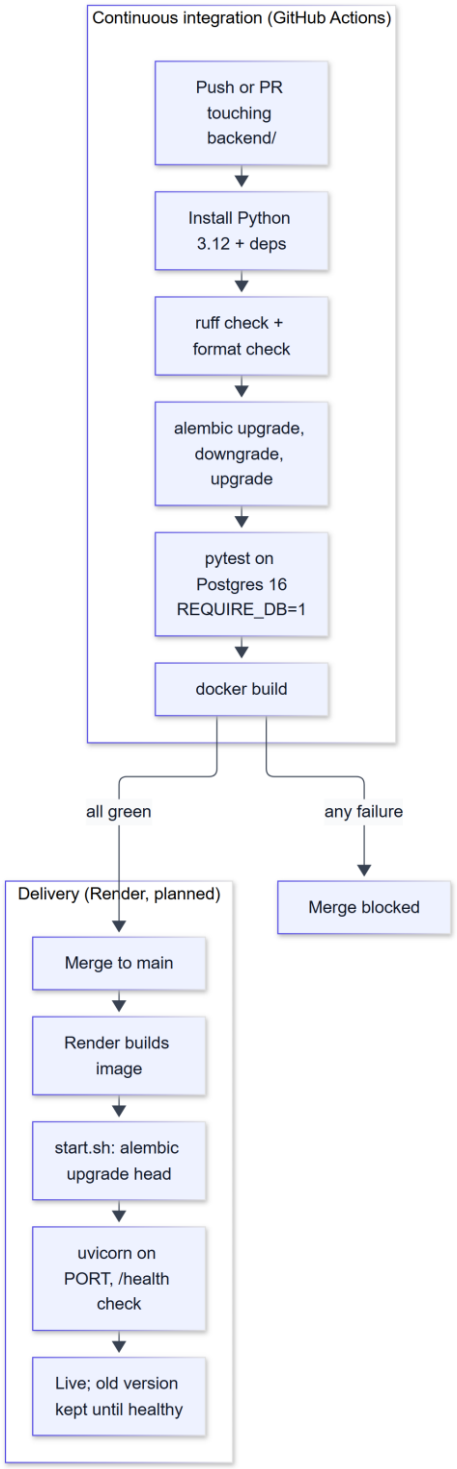


Figure 31. *CI/CD pipeline*

GitHub Actions runs on every push or pull request touching `backend/`: install Python 3.12 and dependencies; ruff lint and format check; migrations upgrade → downgrade → upgrade; tests on a PostgreSQL 16 service container with `REQUIRE_DB=1`; and a Docker build of the production image. Any failure blocks the merge.

9.5 Manual verification

During the build, the live web portal was run against the API with the demo account, and the Dashboard, Activity, Workouts, Connected Devices and Privacy pages were checked in a browser. A second account was used to verify a share end to end: steps readable, heart rate refused with "doesn't include vitals", both entries visible in the owner's access log.

10 Deployment and operations

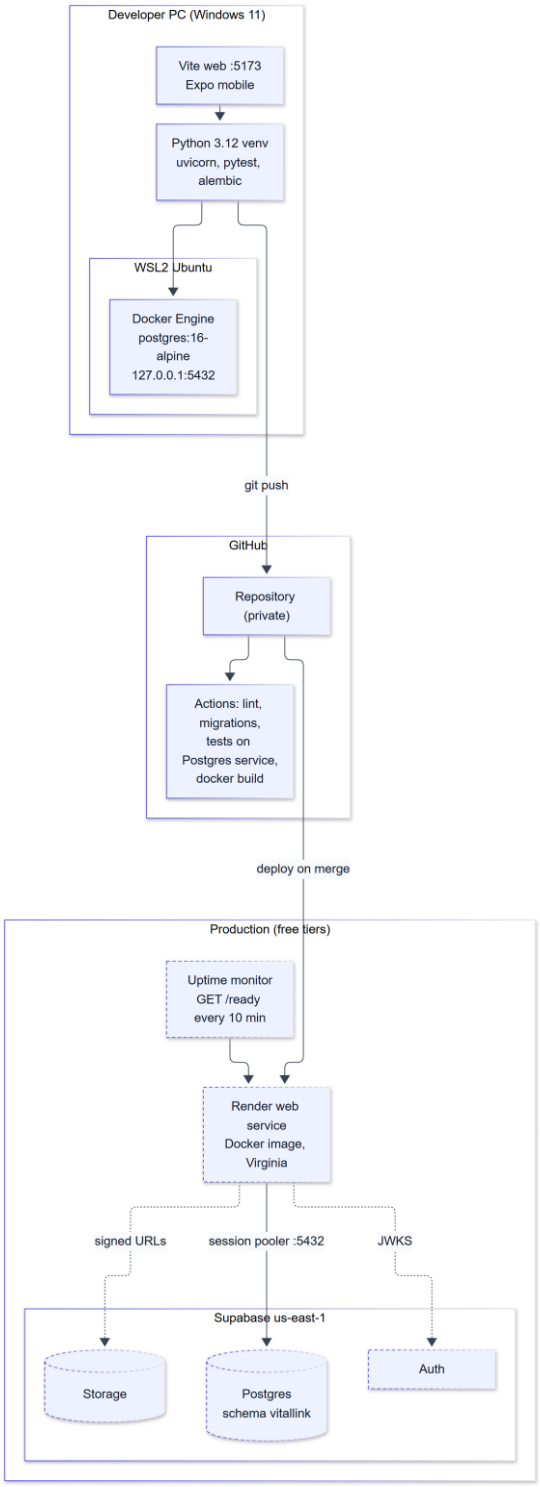


Figure 32. *Deployment diagram (dashed: production, pending accounts)*

10.1 Environments

Table 62. *Environments*

Environment	Runs on	Database	Auth	Status
Local	Python venv on Windows 11; PostgreSQL 16 in Docker inside WSL2 Ubuntu	vitallink + vitallink_test	Dev tokens	Running
CI	GitHub Actions runner	PostgreSQL service container	Dev tokens	Configured
Production	Render free web service (Virginia)	Supabase free, us-east-1, session pooler :5432	Supabase Auth	Needs accounts

10.2 Configuration

Table 63. *Configuration variables*

Variable	Meaning	Where
ENV	local, test or production	Each environment
DATABASE_URL	postgresql+asyncpg://… (use 127.0.0.1, not localhost, on Windows)	.env locally; Render secret
CORS_ORIGIN	The web portal's origin	Plain variable
SUPABASE_URL	Token issuer and JWKS location	Render
JWT_AUDIENCE	Expected audience (authenticated)	Default
DEV*JWT*SECRET	Signs dev tokens; refused in production	.env only
POSTGRES_USER / PASSWORD / DB / PORT	Local container settings	.env only

10.3 Running locally, step by step

```
cd backend
cp .env.example .env                        # set a password and DEV_JWT_SECRET
py -3.12 -m venv .venv
.venv/Scripts/python -m pip install -e ".[dev]"
wsl -d Ubuntu -e sh -c "cd /mnt/c/.../vitallink-app/backend && docker compose up"
.venv/Scripts/python -m alembic upgrade head    # apply migrations 0001-0006
.venv/Scripts/python -m seeds.demo              # demo@example.com, 90 days
.venv/Scripts/python -m uvicorn app.main:app --reload    # http://localhost:8000/docs
.venv/Scripts/python -m app.devtoken demo@example.com    # a bearer token
.venv/Scripts/python -m pytest                  # 134 tests
```

- Keep the WSL terminal open: WSL stops when nothing uses it, and the database stops with it.
- Use 127.0.0.1 in DATABASE_URL on Windows: `localhost` tries IPv6 first and cost 2 seconds per connection (29 ms with 127.0.0.1); the test suite went from minutes to about 10 seconds.
- To see the web portal on live data: start it with `VITE_API_URL=http://localhost:8000` and store a dev token in the browser's `sessionStorage["vl.token"]`.

10.4 Container

- `Dockerfile`: Python 3.12 slim; dependencies in their own layer; runs as a non-root `api` user; health check on /health; .env, tests and docs excluded (image 298 MB).

- `start.sh`: `alembic upgrade head`, then uvicorn on `$PORT`. Render's free tier has no pre-deploy step, so a failed migration stops the new container and Render keeps serving the old version.
- `.gitattributes` pins `*.sh` to LF line endings so the script runs in Linux after a Windows checkout.

10.5 Production rollout (pending)

1. Create shared group accounts (not one person's): Supabase project in us-east-1, Render, GitHub secrets.
2. Supabase: switch the Data API off; require email confirmation; enable TOTP MFA; create one private Storage bucket (25 MB cap, type allow-list).
3. Render: web service from backend/Dockerfile with ENV=production, DATABASE*URL (session pooler :5432), SUPABASE*URL, CORS_ORIGIN.
4. An external free uptime monitor calls /ready every 10 minutes during Dallas daytime, keeping Render awake and Supabase active.
5. Set VITE*API*URL and EXPO*PUBLIC*API_URL in the clients and replace the sign-in stubs with Supabase Auth.

10.6 Observability and free-tier limits

Table 64. *Free-tier limits*

Limit	Our load	Verdict
Render: 512 MB RAM, sleeps after 15 min idle	App about 150 MB	Fits; uptime ping keeps it warm
Render: 750 instance-hours a month	Awake about 17 h a day ≈ 520 h	Fits
Supabase: 500 MB database	About 13.5 MB per user-year	About 30 user-years
Supabase: 1 GB storage, 5 GB egress	Lab PDFs 0.2–2 MB; dashboard about 5 KB	Fits
Supabase: pauses after 7 days idle	/ready touches the database	Fits
GitHub Actions: 2,000 min/month (private)	CI about 5 min per push	Fits; uptime ping runs outside Actions

Request ids appear in a response header and every error body; logs carry route, status, timing and user id, never request bodies. Once live, the team should watch the error rate, requests slower than 1 second, database size against 500 MB, refused-access counts per user (a spike means probing) and failed nightly runs. Backups (planned): a weekly encrypted pg_dump kept 90 days as a private workflow artifact.

11 Implementation status and plan

11.1 Work completed

Table 65. *Commits on main, newest first*

Commit	Date	Change
aff06bc	2026-10-06	Phase 5b: nutrition (migration 0006)
012b37d	2026-10-06	Phase 5a: training (migration 0005)
87355f4	2026-10-06	Phase 4: sharing and the consent engine (migration 0004)
886f132	2026-10-06	Phase 3: health core (migration 0003) and the demo seed
ff87a5b	2026-10-06	Phase 2: accounts and token checking (migration 0002)
5953577	2026-10-06	Dockerfile for Render and the CI workflow
c218f05	2026-10-06	Database connection, /ready, lockdown and audit log (migration 0001)
13efa61	2026-10-06	FastAPI skeleton, error format, local Postgres configuration
f525bca	2026-10-06	Backend design documents and build plan

11.2 Build plan

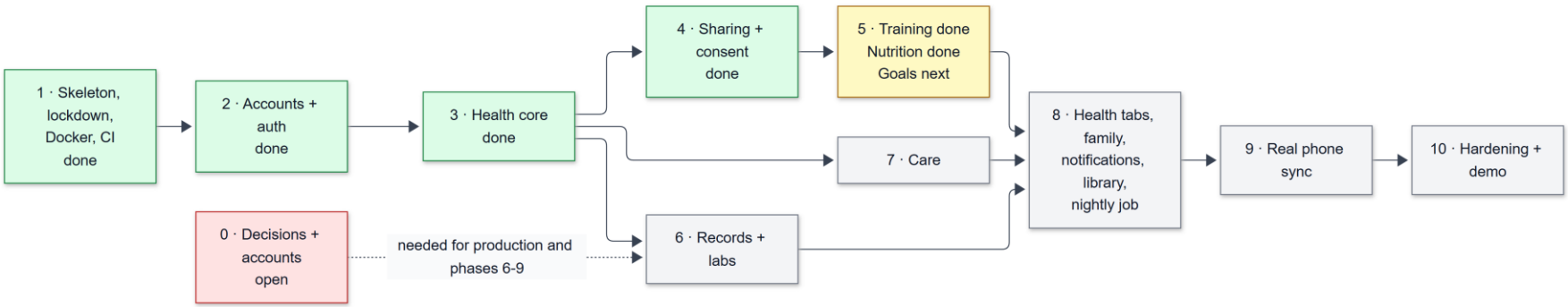


Figure 33. *Build plan: phases and dependencies*

Table 66. *Phases*

Phase	Content	Status
0	Owner decisions; shared Supabase, Render and GitHub accounts	Open
1	Skeleton, lockdown, Docker, CI	Done
2	Accounts and authentication	Done
3	Health core and sync contract	Done
4	Sharing and consent	Done
5	Training (done), nutrition (done), goals and streaks (next)	In progress
6	Records, signed uploads, lab panels	Planned
7	Medications and doses, appointments, conditions, emergency ID	Planned
8	My Health tabs, family view, notifications, library, nightly job	Planned
9	Real HealthKit and Health Connect sync from a development build	Planned
10	Hardening, load test, restore drill, generated client types, demo	Planned

Definition of done for every endpoint: the response matches `types.ts`; owner queries filter by the token; constraints and indexes ship in the same migration; it has rule, query and API tests; and the frontend's `pending()` call is switched to live and checked in the browser.

11.3 Risks

Table 67. *Risk register*

Risk	Likelihood	Impact	Mitigation
A Supabase setting changed by hand exposes tables	Low	Every user's data readable	Four locks in migration 0001; live-project check in CI (planned)
Server asleep during the demo	High (free tier)	30–50 s first load	Uptime ping; open the app 5 minutes before presenting
Database paused before a deadline	Medium	API down	Same ping; check the day before
iOS HealthKit on a free Apple ID	Medium	iPhone sync slips	Verify early in phase 9; Android has no such limit
Free-tier terms change	Medium over a year	Forced move	Nothing provider-specific; move by configuration
Misleading insights	Medium	Wrong conclusions	14-day minimum, correction, association wording
Google Fit APIs end late 2026	Certain	Android sync on Fit would stop	Built on Health Connect from the start

11.4 Open decisions

Table 68. *Decisions awaiting the owner*

#	Question	Recommendation	Blocks
D1	Messages with clinicians	Leave out of v1: inbound clinical messages break consumers-only	Nothing
D2	Emergency QR scanned without signing in	The QR holds the emergency text itself, made on the phone; no public link	Phase 7

#	Question	Recommendation	Blocks
D3	Child profile without an account	v1 Family = people who share with me; dependents later	Phase 8
D4	Share kinds hospital and app	Remove both (applied in the API; Figma still shows them)	Confirmation
D6	How lab results get in	Manual entry form (needs a Figma frame) + demo seed	Phase 6
D8	Digital wellbeing on iOS	Android only: Apple keeps Screen Time data on the device	Phase 9
D9	Remote push	Local notifications only: iOS push needs the paid Apple program	Phase 7
D11	km vs miles	Store SI; screens follow Settings › Units	Frontend
D16	"HIPAA-aligned" copy	Remove it; the app falls under the FTC rule	Frontend

Appendix A Metric catalogue

The 44 metrics the backend accepts. Ranges are per 5-minute bucket; normal ranges drive the dashboard's range chip.

Table 69. *Metric catalogue (migration 0003)*

Id	Code	Scope	Unit	Agg.	Plausible range	Manual	Normal range
1	heart_rate	vitals	bpm	avg	25–250	yes	50–100
2	resting*heart*rate	vitals	bpm	avg	25–150		40–100
3	hrv	vitals	ms	avg	1–300		
4	bp_systolic	vitals	mmHg	avg	60–260	yes	
5	bp_diastolic	vitals	mmHg	avg	30–160	yes	
6	oxygen_saturation	vitals	%	avg	50–100	yes	95–100
7	respiratory_rate	vitals	breaths/min	avg	4–60		12–20
8	glucose	vitals	mg/dL	avg	20–600	yes	70–140
9	body_temperature	vitals	°C	avg	30–45	yes	36.1–37.5
10	vo2_max	vitals	mL/kg/min	last	10–90		
11	hr*high*event	vitals	count	sum	0–10		
12	hr*low*event	vitals	count	sum	0–10		
13	irregular*rhythm*event	vitals	count	sum	0–10		
20	weight	vitals	kg	avg	20–400	yes	
21	body_fat	vitals	%	avg	2–75	yes	
22	lean_mass	vitals	kg	avg	10–200	yes	
23	waist	vitals	cm	avg	10–250	yes	
24	hip	vitals	cm	avg	10–250	yes	
25	chest	vitals	cm	avg	10–250	yes	
26	arm	vitals	cm	avg	10–250	yes	
27	thigh	vitals	cm	avg	10–250	yes	
40	steps	activity	count	sum	0–5000		
41	distance	activity	m	sum	0–6000		
42	active_energy	activity	kcal	sum	0–250		
43	basal_energy	activity	kcal	sum	0–50		
44	exercise_minutes	activity	min	sum	0–5		
45	stand_hours	activity	count	sum	0–1		
46	flights_climbed	activity	count	sum	0–100		
47	mindful_minutes	activity	min	sum	0–5	yes	
48	mood	activity	score	avg	1–5	yes	
49	stress	activity	score	avg	1–5	yes	

Id	Code	Scope	Unit	Agg.	Plausible range	Manual	Normal range
50	energy	activity	score	avg	1–5	yes	
51	screen_time	activity	min	sum	0–5		
52	screen*time*social	activity	min	sum	0–5		
53	screen*time*entertainment	activity	min	sum	0–5		
54	screen*time*productivity	activity	min	sum	0–5		
55	phone_pickups	activity	count	sum	0–200		
56	notifications_received	activity	count	sum	0–1000		
60	sleep_asleep	sleep	min	sum	0–5		
61	sleep_deep	sleep	min	sum	0–5		
62	sleep_core	sleep	min	sum	0–5		
63	sleep_rem	sleep	min	sum	0–5		
64	sleep_awake	sleep	min	sum	0–5		

Appendix B Exercise and food catalogues

Table 70. *Exercise catalogue (migration 0005)*

Id	Exercise	Primary muscle
1	Back Squat	legs
2	Front Squat	legs
3	Leg Press	legs
4	Lunge	legs
5	Romanian Deadlift	legs
6	Calf Raise	legs
7	Deadlift	back
8	Barbell Row	back
9	Pull-up	back
10	Lat Pulldown	back
11	Seated Cable Row	back
12	Bench Press	chest
13	Incline Bench Press	chest
14	Dumbbell Fly	chest
15	Push-up	chest
16	Dip	chest
17	Overhead Press	shoulders
18	Lateral Raise	shoulders
19	Face Pull	shoulders
20	Bicep Curl	arms
21	Hammer Curl	arms
22	Tricep Extension	arms
23	Skull Crusher	arms
24	Plank	core
25	Hanging Leg Raise	core
26	Russian Twist	core
27	Hip Thrust	glutes
28	Glute Bridge	glutes
29	Kettlebell Swing	full_body
30	Burpee	full_body

Table 71. *Food catalogue, per 100 g (migration 0006)*

Food	kcal	Protein g	Carbs g	Fat g	Fibre g	Serving g
Greek yoghurt	73	9.9	3.9	2	0	170
Rolled oats	379	13.2	67.7	6.5	10.1	40
Blueberries	57	0.7	14.5	0.3	2.4	75
Honey	304	0.3	82.4	0	0.2	21
Chicken breast, cooked	165	31	0	3.6	0	120

Food	kcal	Protein g	Carbs g	Fat g	Fibre g	Serving g
White rice, cooked	130	2.7	28.2	0.3	0.4	160
Brown rice, cooked	123	2.7	25.6	1	1.6	160
Avocado	160	2	8.5	14.7	6.7	70
Mixed greens	20	1.8	3.3	0.3	1.9	60
Whey protein powder	400	80	8	6	0	30
Banana	89	1.1	22.8	0.3	2.6	118
Salmon, cooked	206	22.1	0	12.4	0	140
Sweet potato, baked	90	2	20.7	0.2	3.3	150
Broccoli, cooked	35	2.4	7.2	0.4	3.3	90
Egg	143	12.6	0.7	9.5	0	50
Whole wheat bread	247	13	41	3.4	7	32
Peanut butter	588	25	20	50	6	32
Apple	52	0.3	13.8	0.2	2.4	180
Almonds	579	21.2	21.6	49.9	12.5	28
Milk, 2%	50	3.3	4.8	2	0	240
Cheddar cheese	403	24.9	1.3	33.1	0	28
Black beans, cooked	132	8.9	23.7	0.5	8.7	130
Pasta, cooked	158	5.8	30.9	0.9	1.8	140
Ground beef, 90% lean, cooked	217	26.1	0	11.7	0	100
Tofu, firm	144	15.8	2.8	8.7	2.3	100
Olive oil	884	0	0	100	0	14
Orange	47	0.9	11.8	0.1	2.4	140
Spinach	23	2.9	3.6	0.4	2.2	30
Flour tortilla	304	8	50	8	3	45
Hummus	166	7.9	14.3	9.6	6	30

Appendix C Sample requests

C.1 Sync upload

```
POST /v1/me/health/ingest
Authorization: Bearer <token>

{"idempotency_key": "0192a3f4-7c1e-7d2a-9b3c-5e6f7a8b9c0d",
 "platform": "healthkit",
 "devices": [{"external_id": "watch-7F3A", "name": "Apple Watch Series 9", "kind": "wearable",
              "metrics": ["heart_rate", "steps", "active_energy", "sleep_deep"]}],
 "buckets": [{"metric": "steps", "start": "2026-09-13T08:40:00-05:00", "value": 412, "samples":
3},
             {"metric": "heart_rate", "start": "2026-09-13T08:40:00-05:00", "value": 71.5,
"samples": 60}]}

202 Accepted
{"batch_id": 18234, "submitted": 2, "inserted": 1, "updated": 1, "unchanged": 0, "replayed":
false}
```

C.2 Create a share and read through it

```
POST /v1/me/shares                          (owner)
{"email": "susan.carter@example.com", "label": "Susan", "kind": "family",
 "scopes": ["activity", "sleep"], "purpose": "Family check-in"}
201 Created

GET /v1/users/<owner id>/health/series/steps?days=3      (viewer)
200 OK  {"metric": "steps", "unit": "count", "interval": "day", "points": [...]}

GET /v1/users/<owner id>/health/series/heart_rate?days=1 (viewer)
403 {"error": {"code": "scope_not_granted", "message": "This share doesn't include vitals
data.", ...}}
```

C.3 Log a meal

```
POST /v1/me/meals
{"slot": "breakfast", "eaten_at": "2026-10-06T07:10:00-05:00",
 "items": [{"food_id": "<Greek yoghurt id>", "quantity_g": 200},
           {"label": "Coffee with milk", "quantity_g": 250, "kcal": 40,
            "protein_g": 2, "carbs_g": 3, "fat_g": 2}]}
201 Created
{"id": "...", "name": "Breakfast", "items": ["Greek yoghurt", "Coffee with milk"],
 "kcal": 186, "protein_g": 22, "carbs_g": 11, "fat_g": 6, ...}
```